Metodología Exhaustiva para Ensemble Forecasting Espacio-Temporal: Especificación Técnica Completa

Índice

1. Marco Teórico Fundamental **4**
- 1.1. Definiciones Formales . . . 4
- 1.2. Modelo Generador Espacio-Temporal . . . 4
- 1.3. Teoría de Ensemble Óptimo . . . 4

2. Arquitectura del Sistema **5**
- 2.1. Especificación de Componentes . . . 5
- 2.2. Estructura de Repositorio Obligatoria . . . 5

3. Plan de Formación Técnica **6**
- 3.1. Módulo 1: Fundamentos Matemáticos (4 semanas) . . . 6
  - 3.1.1. Semana 1: Álgebra Lineal Aplicada . . . 6
  - 3.1.2. Semana 2: Procesos Estocásticos . . . 6
  - 3.1.3. Semana 3: Series Temporales Univariadas . . . 6
  - 3.1.4. Semana 4: Modelos VAR y Cointegración . . . 7
- 3.2. Módulo 2: Econometría Espacial (3 semanas) . . . 7
  - 3.2.1. Semana 5: Dependencia Espacial . . . 7
  - 3.2.2. Semana 6: Modelos SAR y SEM . . . 8
  - 3.2.3. Semana 7: Modelos Espacio-Temporales . . . 8
- 3.3. Módulo 3: Machine Learning Avanzado (3 semanas) . . . 9
  - 3.3.1. Semana 8: Ensemble Methods . . . 9
  - 3.3.2. Semana 9: Deep Learning para Series Temporales . . . 9
  - 3.3.3. Semana 10: Meta-Learning . . . 10

4. Validación y Evaluación **10**
- 4.1. Validación Temporal Rigurosa . . . 10
- 4.2. Métricas de Evaluación . . . 10
  - 4.2.1. Métricas de Precisión . . . 10
  - 4.2.2. Coherencia Espacial . . . 10
- 4.3. Tests Estadísticos . . . 11
  - 4.3.1. Test de Diebold-Mariano . . . 11

5. Implementación de Modelos Base **11**
- 5.1. Modelos Autorregresivos Espaciales . . . 11
- 5.2. Modelos VAR Espaciales . . . 13

6. Optimización de Ensemble **14**
6.1. Ensemble Estático Óptimo . . . 14
6.2. Ensemble Dinámico con Online Learning . . . 15

7. Cronograma de Implementación **16**
7.1. Fase 1: Fundamentos (Semanas 1-4) . . . 16
7.1.1. Semana 1: Setup y Infraestructura . . . 16
7.1.2. Semana 2: Utilidades Espaciales . . . 17
7.1.3. Semana 3: Modelos Temporales Base . . . 18
7.1.4. Semana 4: Validación Temporal . . . 18
7.2. Fase 2: Modelos Espaciales (Semanas 5-7) . . . 19
7.2.1. Semana 5: Tests de Dependencia Espacial . . . 19
7.2.2. Semana 6: Modelos SAR/SEM . . . 19
7.2.3. Semana 7: Modelos Panel Espacial . . . 22
7.3. Fase 3: Machine Learning y Ensemble (Semanas 8-10) . . . 23
7.3.1. Semana 8: Modelos ML Base . . . 23
7.3.2. Semana 9: Arquitectura LSTM Espacial . . . 24
7.3.3. Semana 10: Ensemble Orchestrator . . . 25

8. Métricas de Evaluación Avanzadas **28**
8.1. Métricas Específicas Espacio-Temporales . . . 28
8.1.1. Coherencia Espacial Generalizada . . . 28
8.1.2. Persistencia Temporal de Errores . . . 28
8.2. Tests de Superior Predictive Ability . . . 28

9. Optimización Avanzada y Regularización **31**
9.1. Optimización Bayesiana de Hiperparámetros . . . 31
9.2. Regularización Espacial . . . 35

10. Monitoring y Drift Detection **36**
10.1. Detección de Drift Multivariado . . . 36
10.2. Sistema de Alertas y Reentrenamiento . . . 39

11. Criterios de Éxito y Benchmarking **44**
11.1. KPIs Técnicos Obligatorios . . . 44
11.2. Tests de Aceptación . . . 45

12. Deployment y Productización **50**
12.1. Containerización . . . 50
12.2. API REST Completa . . . 51

13. Bibliografia **59**

14. Anexo A: Checklist de Implementación **61**
14.1. Checklist Técnico Obligatorio . . . 61
14.2. Criterios de Calidad No Negociables . . . 62

15. Anexo B: Cronograma Detallado **63**
15.1. Timeline Realista (20 semanas) . . . 63
15.2. Hitos Críticos y Gates de Calidad . . . 64

16. Anexo C: Resources y Budget **65**
16.1. Presupuesto Detallado 65
16.2. ROI Estimado 66

17. Consideraciones Finales **66**
17.1. Advertencias Críticas 66
17.2. Éxito Garantizado 67

1. Marco Teórico Fundamental

1.1. Definiciones Formales

Sea $(\Omega, \mathcal{F}, \mathbb{P})$ un espacio de probabilidad completo. Definimos el proceso espacio-temporal como:

Definición 1 (Proceso Espacio-Temporal)**.** Un proceso estocástico $\{Y_{i,t}(\omega) : i \in \mathcal{S}, t \in \mathcal{T}, \omega \in \Omega\}$ donde:

- $\mathcal{S} = \{s_1, s_2, \ldots, s_N\} \subset \mathbb{R}^2$ conjunto de ubicaciones espaciales
- $\mathcal{T} = \{1, 2, \ldots, T\}$ dominio temporal discreto
- $Y_{i,t} \in \mathbb{R}^p$ vector de variables observadas

Definición 2 (Matriz de Ponderación Espacial)**.** Una matriz $W \in \mathbb{R}^{N \times N}$ es una matriz de ponderación espacial válida si:

$$w_{ii} = 0 \quad \forall i \in \{1, \ldots, N\} \tag{1}$$

$$w_{ij} \geq 0 \quad \forall i, j \in \{1, \ldots, N\} \tag{2}$$

$$\sum_{j=1}^{N} w_{ij} = 1 \quad \forall i \text{ (row-standardized)} \tag{3}$$

1.2. Modelo Generador Espacio-Temporal

El proceso generador de datos sigue la especificación:

$$\mathbf{Y}_{i,t} = \boldsymbol{\mu}_{i,t} + \rho \sum_{j=1}^{N} w_{ij} \mathbf{Y}_{j,t} + \sum_{k=1}^{p} \boldsymbol{\Phi}_k \mathbf{Y}_{i,t-k} + \boldsymbol{\varepsilon}_{i,t} \tag{4}$$

donde:

- $\boldsymbol{\mu}_{i,t} \in \mathbb{R}^d$ componente determinística
- $\rho \in (-1, 1)$ parámetro de dependencia espacial
- $\boldsymbol{\Phi}_k \in \mathbb{R}^{d \times d}$ matrices de coeficientes autorregresivos
- $\boldsymbol{\varepsilon}_{i,t} \sim \text{iid } \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$

1.3. Teoría de Ensemble Óptimo

Teorema 1 (Ensemble Óptimo de Hansen)**.** Para función de pérdida cuadrática $L(\mathbf{y}, \hat{\mathbf{y}}) = |\mathbf{y} - \hat{\mathbf{y}}|^2$, el ensemble óptimo está dado por:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \mathbb{E}[L(\mathbf{y}, \mathbf{F}\mathbf{w})] \tag{5}$$

sujeto a $\sum_{m=1}^{M} w_m = 1$ y $w_m \geq 0$.

Demostración. La solución analítica viene dada por:

$$\mathbf{w}^* = (\mathbf{F}^T \mathbf{F})^{-1} \mathbf{F}^T \mathbf{y} \tag{6}$$

donde $\mathbf{F} = [\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \ldots, \hat{\mathbf{y}}_M]$ es la matriz de predicciones de los modelos base. □

2. Arquitectura del Sistema

2.1. Especificación de Componentes

El sistema se compone de los siguientes módulos críticos:

Algorithm 1 Pipeline Principal del Ensemble

Input: Datos históricos $\{Y_{i,t}\}$, matriz espacial W, horizonte h
Preprocesamiento:
  Validar estacionariedad por región
  Construir features espaciales y temporales
  Detectar y tratar outliers espacio-temporales
Entrenamiento Modelos Base:
for $m = 1$ to M **do**
  Entrenar modelo f_m con validación temporal
  Calcular predicciones out-of-sample $\hat{Y}_{i,t}^{(m)}$
end for
Optimización Ensemble:
Resolver problema de optimización (5)
Obtener pesos óptimos $\mathbf{w}^*$
Predicción:
$\hat{Y}_{i,t+h} = \sum_{m=1}^{M} w_m^* f_m(X_{i,t})$
return Predicciones ensemble con intervalos de confianza

2.2. Estructura de Repositorio Obligatoria

La estructura del proyecto debe seguir estrictamente:

```
ensemble_forecaster/
          src/
                config/
                      base_config.py
                      model_configs.py
                data/
                      loaders.py
                      preprocessors.py
                      spatial_utils.py
                models/
                      temporal/
                      spatial/
                      ml/
                ensemble/
                evaluation/
          tests/
          docs/
          configs/
```

3. Plan de Formación Técnica

3.1. Módulo 1: Fundamentos Matemáticos (4 semanas)

3.1.1. Semana 1: Álgebra Lineal Aplicada

Referencias obligatorias:

- **(author?)** (17) Capítulos 1-6
- **(author?)** (18) Capítulos 1-3

Implementaciones requeridas:

```python
def svd_decomposition(A: NDArray[np.float64]) -> Tuple[NDArray, NDArray, NDArray]:
    """Implementaci n SVD desde cero con verificaci n num rica."""

def condition_number(A: NDArray[np.float64]) -> float:
    """C lculo del n mero de condici n con manejo de casos l mite."""

def solve_linear_system(A: NDArray[np.float64],
                        b: NDArray[np.float64]) -> NDArray[np.float64]:
    """Resolver Ax = b con pivoting parcial."""
```

3.1.2. Semana 2: Procesos Estocásticos

Referencias críticas:

- **(author?)** (16) Capítulos 4-6
- **(author?)** (8) Capítulos 9-10

Conceptos no-negociables:

1. Martingalas y submartingalas
2. Teoremas ergódicos
3. Convergencia estocástica

3.1.3. Semana 3: Series Temporales Univariadas

Literatura fundamental:

- **(author?)** (9) Capítulos 1-5, 20-21
- **(author?)** (3) Capítulos 1-9

Implementación obligatoria de modelo ARIMA completo:

```
class ARIMAModel:
    def __init__(self, order: Tuple[int, int, int]) -> None:
        self.p, self.d, self.q = order

    def fit(self, y: NDArray[np.float64]) -> 'ARIMAModel':
        """Maximum likelihood estimation con gradiente anal tico
           ."""

    def predict(self, steps: int) -> Tuple[NDArray[np.float64],
                                           NDArray[np.float64]]:
        """Predicci n con intervalos de confianza."""

    def ljung_box_test(self) -> Tuple[float, float]:
        """Test de autocorrelaci n residual."""
```

3.1.4. Semana 4: Modelos VAR y Cointegración

Referencias esenciales:

- **(author?)** (14) Capítulos 1-7
- **(author?)** (12)

Implementación crítica del test de Johansen:

$$\lambda_{\text{trace}}(r) = -T \sum_{i=r+1}^{g} \ln(1 - \hat{\lambda}_i) \tag{7}$$

$$\lambda_{\max}(r, r+1) = -T \ln(1 - \hat{\lambda}_{r+1}) \tag{8}$$

3.2. Módulo 2: Econometría Espacial (3 semanas)

3.2.1. Semana 5: Dependencia Espacial

Referencias obligatorias:

- **(author?)** (1)
- **(author?)** (5)

Implementación del estadístico I de Moran:

$$I = \frac{N}{\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij}(y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2} \tag{9}$$

```
def morans_i(y: NDArray[np.float64],
             W: NDArray[np.float64]) -> Tuple[float, float]:
    """C lculo de I de Moran con test de significancia."""
    n = len(y)
    y_centered = y - np.mean(y)
```

```
    numerator = np.sum(W * np.outer(y_centered, y_centered))
    denominator = np.sum(y_centered**2)

    W_sum = np.sum(W)
    I = (n / W_sum) * (numerator / denominator)

    # Varianza bajo H0
    E_I = -1 / (n - 1)
    var_I = compute_moran_variance(W, n)

    z_score = (I - E_I) / np.sqrt(var_I)
    p_value = 2 * (1 - stats.norm.cdf(np.abs(z_score)))

    return I, p_value
```

3.2.2. Semana 6: Modelos SAR y SEM

Literatura core:

- **(author?)** (13) Capítulos 2-4
- **(author?)** (7)

Modelo SAR con estimación ML:

$$\mathbf{y} = \rho W \mathbf{y} + X\boldsymbol{\beta} + \boldsymbol{\varepsilon} \tag{10}$$

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_N) \tag{11}$$

La función de log-verosimilitud es:

$$\ln L(\rho, \boldsymbol{\beta}, \sigma^2) = -\frac{N}{2}\ln(2\pi\sigma^2) + \ln|I_N - \rho W| - \frac{1}{2\sigma^2}\boldsymbol{\varepsilon}^T\boldsymbol{\varepsilon} \tag{12}$$

3.2.3. Semana 7: Modelos Espacio-Temporales

Referencias avanzadas:

- **(author?)** (6)
- **(author?)** (22)

Panel espacial dinámico:

$$y_{it} = \tau y_{it-1} + \rho \sum_{j=1}^{N} w_{ij} y_{jt} + x_{it}^T \boldsymbol{\beta} + \mu_i + \varepsilon_{it} \tag{13}$$

3.3. Módulo 3: Machine Learning Avanzado (3 semanas)

3.3.1. Semana 8: Ensemble Methods

Papers fundamentales:

- **(author?)** (2)
- **(author?)** (4)
- **(author?)** (20)

Implementación de Stacked Generalization:

```
class StackingEnsemble:
    def __init__(self, base_models: List[BaseModel],
                 meta_learner: BaseModel,
                 cv_folds: int = 5) -> None:
        self.base_models = base_models
        self.meta_learner = meta_learner
        self.cv = TimeSeriesSplit(n_splits=cv_folds)

    def _generate_meta_features(self, X: NDArray, y: NDArray) ->
       NDArray:
        meta_features = np.zeros((len(X), len(self.base_models)))

        for fold_idx, (train_idx, val_idx) in enumerate(self.cv.
           split(X)):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train = y[train_idx]

            for model_idx, model in enumerate(self.base_models):
                model_copy = clone(model)
                model_copy.fit(X_train, y_train)
                meta_features[val_idx, model_idx] = model_copy.
                   predict(X_val)

        return meta_features
```

3.3.2. Semana 9: Deep Learning para Series Temporales

Arquitecturas obligatorias:

- LSTM espacio-temporal
- Transformer con atención espacial
- N-BEATS con componentes espaciales

Referencias:

- **(author?)** (10)
- **(author?)** (19)
- **(author?)** (15)

3.3.3. Semana 10: Meta-Learning

Literatura especializada:

- **(author?)** (11)
- **(author?)** (21)

4. Validación y Evaluación

4.1. Validación Temporal Rigurosa

Para series temporales, la validación cruzada estándar viola la estructura temporal. Implementamos validación forward-chaining:

Algorithm 2 Purged Group Time Series Split

1: **Input:** Datos X, grupos temporales G, gap g
2: **for** $i = 1$ to n_{splits} **do**
3: train_end = start + $i \times$ step
4: test_start = train_end + $g + 1$
5: test_end = test_start + test_size
6: Eliminar observaciones en gap para evitar data leakage (train_indices, test_indices)
7: **end for**

4.2. Métricas de Evaluación

4.2.1. Métricas de Precisión

$$\text{RMSE}_{i,h} = \sqrt{\frac{1}{T-h}\sum_{t=h+1}^{T}(y_{i,t} - \hat{y}_{i,t|t-h})^2} \tag{14}$$

$$\text{MASE}_{i,h} = \frac{\text{MAE}_{i,h}}{\frac{1}{T-1}\sum_{t=2}^{T}|y_{i,t} - y_{i,t-1}|} \tag{15}$$

$$\text{DA}_{i,h} = \frac{1}{T-h}\sum_{t=h+1}^{T}\mathbb{I}[\text{sign}(\Delta y_{i,t}) = \text{sign}(\Delta \hat{y}_{i,t|t-h})] \tag{16}$$

4.2.2. Coherencia Espacial

Definimos el índice de coherencia espacial como:

$$\text{SC}_t = 1 - \frac{\sum_{i,j} w_{ij}|\varepsilon_{i,t} - \varepsilon_{j,t}|}{\sum_{i,j} w_{ij}(|\varepsilon_{i,t}| + |\varepsilon_{j,t}|)} \tag{17}$$

donde $\varepsilon_{i,t}$ son los errores de predicción.

4.3. Tests Estadísticos

4.3.1. Test de Diebold-Mariano

Para comparar accuracy de dos modelos:

$$DM = \frac{\bar{d}}{\sqrt{\hat{\gamma}_0/T}} \tag{18}$$

donde $\bar{d}$ es la diferencia promedio de pérdidas y $\hat{\gamma}_0$ es la varianza de largo plazo.

```
def diebold_mariano_test(errors1: NDArray[np.float64],
                         errors2: NDArray[np.float64],
                         horizon: int) -> Tuple[float, float]:
    """
    Test de Diebold-Mariano para comparaci n de accuracy.

    Returns
    -------
    dm_stat : float
        Estad stico DM
    p_value : float
        P-valor bilateral
    """
    loss_diff = errors1**2 - errors2**2
    mean_diff = np.mean(loss_diff)

    # Varianza con correcci n por autocorrelaci n
    gamma_0 = np.var(loss_diff, ddof=1)
    if horizon > 1:
        # Correcci n Newey-West
        gamma_0 = newey_west_variance(loss_diff, horizon-1)

    dm_stat = mean_diff / np.sqrt(gamma_0 / len(loss_diff))
    p_value = 2 * (1 - stats.norm.cdf(np.abs(dm_stat)))

    return dm_stat, p_value
```

5. Implementación de Modelos Base

5.1. Modelos Autorregresivos Espaciales

La implementación del modelo SAR requiere maximizar la log-verosimilitud (12):

```
class SpatialLagModel:
    def __init__(self, W: NDArray[np.float64]) -> None:
        self.W = self._validate_weights(W)
        self.rho: Optional[float] = None
        self.beta: Optional[NDArray[np.float64]] = None
        self.sigma2: Optional[float] = None

```

```
    def _log_likelihood(self, params: NDArray[np.float64],
                        y: NDArray[np.float64],
                        X: NDArray[np.float64]) -> float:
        rho, beta, sigma2 = self._unpack_params(params)
        n = len(y)

        # Jacobiano: |I - rho * W|
        A = np.eye(n) - rho * self.W
        try:
            log_det_A = np.log(np.linalg.det(A))
        except:
            # Alternativa estable: eigenvalores
            eigenvals = np.linalg.eigvals(A)
            log_det_A = np.sum(np.log(eigenvals.real))

        # Transformaci n espacial
        y_transformed = A @ y
        X_transformed = A @ X

        # Residuos
        residuals = y_transformed - X_transformed @ beta

        # Log-verosimilitud
        ll = -0.5 * n * np.log(2 * np.pi * sigma2)
        ll += log_det_A
        ll -= np.sum(residuals**2) / (2 * sigma2)

        return -ll  # Negativo para minimizaci n

    def fit(self, X: NDArray[np.float64],
            y: NDArray[np.float64]) -> 'SpatialLagModel':

        # Bounds para rho basados en eigenvalues de W
        w_eigenvals = np.linalg.eigvals(self.W)
        rho_min = 1.0 / np.min(w_eigenvals.real)
        rho_max = 1.0 / np.max(w_eigenvals.real)

        # Estimaci n por ML
        initial_params = self._get_initial_params(X, y)
        bounds = [(rho_min + 0.001, rho_max - 0.001)] + \
                 [(None, None)] * X.shape[1] + [(1e-6, None)]

        result = minimize(self._log_likelihood, initial_params,
                          args=(y, X), bounds=bounds, method='L-
                             BFGS-B')

        self.rho, self.beta, self.sigma2 = self._unpack_params(
           result.x)
        self.fitted = True

        return self
```

5.2. Modelos VAR Espaciales

Para el modelo VAR espacial:

$$\mathbf{Y}_t = \boldsymbol{\mu} + \rho W \mathbf{Y}_t + \sum_{k=1}^{p} \boldsymbol{\Phi}_k \mathbf{Y}_{t-k} + \boldsymbol{\varepsilon}_t \tag{19}$$

```python
class SpatialVAR:
    def __init__(self, lags: int, W: NDArray[np.float64]) -> None:
        self.lags = lags
        self.W = W
        self.coef_: Optional[NDArray] = None
        self.spatial_rho: Optional[float] = None

    def fit(self, Y: NDArray[np.float64]) -> 'SpatialVAR':
        """
        Estimaci n por 2SLS o ML seg n tama o del sistema.
        """
        T, K = Y.shape

        # Crear matriz de regresores
        X = self._build_design_matrix(Y)

        if K * self.lags > 50:  # Sistema grande
            self._fit_2sls(Y, X)
        else:  # Sistema peque o
            self._fit_ml(Y, X)

        return self

    def impulse_response(self, periods: int,
                         shock_var: int,
                         response_var: int) -> NDArray[np.float64]:
        """
        Funci n de respuesta impulso considerando efectos espaciales.
        """
        K = self.coef_.shape[0]

        # Matriz de coeficientes del VAR
        Phi = self._get_companion_matrix()

        # Multiplicador espacial
        spatial_mult = np.linalg.inv(np.eye(K) - self.spatial_rho * self.W)

        # IRF
        irf = np.zeros(periods)
        shock = np.zeros(K)
```

```
        shock[shock_var] = 1.0

        for t in range(periods):
            response = np.linalg.matrix_power(Phi, t) @
                spatial_mult @ shock
            irf[t] = response[response_var]

        return irf
```

6. Optimización de Ensemble

6.1. Ensemble Estático Óptimo

El problema de optimización para pesos óptimos:

$$\min_{\mathbf{w}} \quad \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w} \tag{20}$$

$$\text{sujeto a} \quad \mathbf{1}^T \mathbf{w} = 1 \tag{21}$$

$$w_i \geq 0, \quad i = 1, \ldots, M \tag{22}$$

donde $\boldsymbol{\Sigma}$ es la matriz de covarianza de errores.

```
from scipy.optimize import minimize
import cvxpy as cp

class OptimalEnsemble:
    def __init__(self, regularization: float = 0.0) -> None:
        self.reg_param = regularization
        self.weights_: Optional[NDArray] = None

    def fit(self, predictions: NDArray[np.float64],
            y_true: NDArray[np.float64]) -> 'OptimalEnsemble':
        """
        Optimizaci n convexa para pesos  ptimos .
        """
        T, M = predictions.shape

        # Matriz de covarianza de errores
        errors = predictions - y_true.reshape(-1, 1)
        Sigma = np.cov(errors.T)

        # Regularizaci n
        Sigma += self.reg_param * np.eye(M)

        # Optimizaci n convexa con CVXPY
        w = cp.Variable(M)
        objective = cp.Minimize(cp.quad_form(w, Sigma))
        constraints = [cp.sum(w) == 1, w >= 0]

        problem = cp.Problem(objective, constraints)
```

```
        problem.solve(solver=cp.ECOS)

        if problem.status not in [cp.OPTIMAL, cp.
            OPTIMAL_INACCURATE]:
            raise ValueError(f"Optimization failed: {problem.
                status}")

        self.weights_ = w.value
        return self

    def predict(self, predictions: NDArray[np.float64]) ->
        NDArray[np.float64]:
        if self.weights_ is None:
            raise ValueError("Ensemble not fitted")

        return predictions @ self.weights_
```

6.2. Ensemble Dinámico con Online Learning

Para adaptación temporal de pesos:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L_t(\mathbf{w}_t) \tag{23}$$

donde $L_t(\mathbf{w}) = (\mathbf{y}_t - \mathbf{f}_t^T \mathbf{w})^2$ es la pérdida instantánea.

```
class OnlineEnsemble:
    def __init__(self, n_models: int, learning_rate: float =
        0.01,
                 forgetting_factor: float = 0.95) -> None:
        self.n_models = n_models
        self.lr = learning_rate
        self.forgetting_factor = forgetting_factor
        self.weights = np.ones(n_models) / n_models
        self.loss_history = []

    def update(self, predictions: NDArray[np.float64],
               y_true: float) -> None:
        """
        Actualizaci n online de pesos v a gradient descent.
        """
        # Error ensemble actual
        ensemble_pred = np.dot(self.weights, predictions)
        error = y_true - ensemble_pred

        # Gradiente de la p rdida cuadr tica
        gradient = -2 * error * predictions

        # Actualizaci n con momentum
        self.weights -= self.lr * gradient

```

```
        # Proyecci n al simplex
        self.weights = self._project_simplex(self.weights)

        # Actualizar historial con forgetting
        self.loss_history.append(error**2)
        if len(self.loss_history) > 1000:
            self.loss_history = self.loss_history[-1000:]

    def _project_simplex(self, w: NDArray[np.float64]) -> NDArray
        [np.float64]:
        """Proyecci n al simplex unitario."""
        if np.sum(w) == 1.0 and np.all(w >= 0):
            return w

        # Algoritmo de Duchi et al.
        w_sorted = np.sort(w)[::-1]
        cumsum_w = np.cumsum(w_sorted)
        lambda_vals = (cumsum_w - 1.0) / np.arange(1, len(w) + 1)

        rho = np.max(np.where(w_sorted > lambda_vals)[0])
        lambda_star = lambda_vals[rho]

        return np.maximum(w - lambda_star, 0.0)
```

7. Cronograma de Implementación

7.1. Fase 1: Fundamentos (Semanas 1-4)

7.1.1. Semana 1: Setup y Infraestructura

Deliverables obligatorios:

- Repositorio con estructura completa
- CI/CD pipeline funcional
- Pre-commit hooks configurados
- Docker containers para desarrollo

Criterios de aceptación:

- Tests unitarios ¿90 % coverage
- Type hints 100 % en código público
- Documentación automática generándose

7.1.2. Semana 2: Utilidades Espaciales

Implementación completa de la clase `SpatialWeights`:

```python
class SpatialWeights:
    @staticmethod
    def contiguity(adjacency: NDArray[np.bool_]) -> NDArray[np.
       float64]:
        """Matriz de contig idad desde adjacencia."""
        W = adjacency.astype(float)
        row_sums = W.sum(axis=1)
        row_sums[row_sums == 0] = 1  # Evitar divisi n por cero
        return W / row_sums[:, np.newaxis]

    @staticmethod
    def distance(coords: NDArray[np.float64],
                 threshold: float,
                 decay: str = 'linear') -> NDArray[np.float64]:
        """Matriz de distancias con threshold."""
        from scipy.spatial.distance import pdist, squareform

        distances = squareform(pdist(coords))

        if decay == 'linear':
            W = np.maximum(0, threshold - distances) / threshold
        elif decay == 'exponential':
            W = np.exp(-distances / threshold)
        else:
            raise ValueError(f"Unknown␣decay␣function:␣{decay}")

        np.fill_diagonal(W, 0)
        return SpatialWeights._row_standardize(W)

    @staticmethod
    def k_nearest(coords: NDArray[np.float64], k: int) -> NDArray
       [np.float64]:
        """K-vecinos m s cercanos."""
        from sklearn.neighbors import NearestNeighbors

        nbrs = NearestNeighbors(n_neighbors=k+1).fit(coords)
        distances, indices = nbrs.kneighbors(coords)

        n = len(coords)
        W = np.zeros((n, n))

        for i in range(n):
            neighbors = indices[i, 1:]  # Excluir self
            neighbor_distances = distances[i, 1:]
            W[i, neighbors] = 1.0 / neighbor_distances

        return SpatialWeights._row_standardize(W)
```

7.1.3. Semana 3: Modelos Temporales Base

Modelos requeridos:

1. ARIMA con estimación ML completa
2. VAR con tests de cointegración
3. VECM si cointegración detectada
4. State Space Models básicos

7.1.4. Semana 4: Validación Temporal

Implementación crítica de validación sin data leakage:

```python
class PurgedTimeSeriesSplit:
    def __init__(self, n_splits: int, test_size: int,
                 gap: int = 0, purge_pct: float = 0.01) -> None:
        self.n_splits = n_splits
        self.test_size = test_size
        self.gap = gap
        self.purge_pct = purge_pct

    def split(self, X: NDArray[np.float64],
              groups: Optional[NDArray] = None) -> Iterator[Tuple
                 [NDArray[np.int64], NDArray[np.int64]]]:

        n_samples = len(X)
        purge_size = int(n_samples * self.purge_pct)

        # Tama o de cada fold
        fold_size = (n_samples - self.test_size - self.gap) //
           self.n_splits

        for i in range(self.n_splits):
            #  ndices de entrenamiento
            train_start = 0
            train_end = train_start + (i + 1) * fold_size

            # Purge: eliminar observaciones cercanas al test
            purge_start = max(0, train_end - purge_size)
            train_indices = np.arange(train_start, purge_start)

            #  ndices de test
            test_start = train_end + self.gap
            test_end = test_start + self.test_size

            if test_end > n_samples:
                break

            test_indices = np.arange(test_start, test_end)

            yield train_indices, test_indices
```

7.2. Fase 2: Modelos Espaciales (Semanas 5-7)

7.2.1. Semana 5: Tests de Dependencia Espacial

Tests obligatorios:

1. Test I de Moran (9)
2. LM test para lag dependence
3. LM test para error dependence
4. Robust LM tests

Implementación del test LM para dependencia espacial:

$$LM_\lambda = \frac{[\mathbf{e}^T W \mathbf{y}]^2}{\mathbf{e}^T\mathbf{e} \cdot \text{tr}(W^T W + W^2)} \tag{24}$$

donde $\mathbf{e}$ son los residuos OLS.

```python
def lm_lag_test(residuals: NDArray[np.float64],
                X: NDArray[np.float64],
                y: NDArray[np.float64],
                W: NDArray[np.float64]) -> Tuple[float, float]:
    """LM test para dependencia espacial en lag."""

    # Residuos OLS
    e = residuals

    # Numerador
    Wy = W @ y
    numerator = (e.T @ Wy)**2

    # Denominador
    sigma2 = np.sum(e**2) / len(e)
    trace_term = np.trace(W.T @ W + W @ W)
    denominator = sigma2 * trace_term

    # Estad stico LM
    lm_stat = numerator / denominator
    p_value = 1 - stats.chi2.cdf(lm_stat, df=1)

    return lm_stat, p_value
```

7.2.2. Semana 6: Modelos SAR/SEM

Implementación completa de Maximum Likelihood para SAR:

```python
class SpatialAutoregressiveModel:
    def __init__(self, W: NDArray[np.float64],
                 method: str = 'ml') -> None:
        self.W = self._validate_spatial_weights(W)
```

```
        self.method = method
        self.rho_: Optional[float] = None
        self.beta_: Optional[NDArray] = None
        self.sigma2_: Optional[float] = None

    def _concentrated_log_likelihood(self, rho: float,
                                     y: NDArray, X: NDArray) ->
                                        float:
        """Log-likelihood concentrada en beta y sigma2."""
        n = len(y)

        # Transformaci n espacial
        A = np.eye(n) - rho * self.W

        # Jacobiano log-determinant
        try:
            log_det_A = np.log(np.linalg.det(A))
        except:
            # M todo estable para matrices grandes
            eigenvals = np.linalg.eigvals(A)
            log_det_A = np.sum(np.log(eigenvals.real + 1e-12))

        # Variables transformadas
        Ay = A @ y
        AX = A @ X

        # Estimadores concentrados
        try:
            beta_conc = np.linalg.solve(AX.T @ AX, AX.T @ Ay)
        except np.linalg.LinAlgError:
            # Regularizaci n si matriz singular
            AXTAXreg = AX.T @ AX + 1e-6 * np.eye(AX.shape[1])
            beta_conc = np.linalg.solve(AXTAXreg, AX.T @ Ay)

        residuals = Ay - AX @ beta_conc
        sigma2_conc = np.sum(residuals**2) / n

        # Log-likelihood concentrada
        ll = -0.5 * n * np.log(2 * np.pi * sigma2_conc)
        ll += log_det_A
        ll -= 0.5 * n

        return ll

    def fit(self, X: NDArray[np.float64],
            y: NDArray[np.float64]) -> '
               SpatialAutoregressiveModel':

        # Bounds para rho
        eigenvals = np.linalg.eigvals(self.W)
        rho_min = 1.0 / np.min(eigenvals.real) + 1e-6
```

```
        rho_max = 1.0 / np.max(eigenvals.real) - 1e-6

        # Optimizaci n unidimensional en rho
        from scipy.optimize import minimize_scalar

        result = minimize_scalar(
            lambda rho: -self._concentrated_log_likelihood(rho, y, X),
            bounds=(rho_min, rho_max),
            method='bounded'
        )

        if not result.success:
            raise RuntimeError("ML estimation failed to converge")

        # Par metros  ptimos
        self.rho_ = result.x

        # Recalcular beta y sigma2
        n = len(y)
        A = np.eye(n) - self.rho_ * self.W
        Ay = A @ y
        AX = A @ X

        self.beta_ = np.linalg.solve(AX.T @ AX, AX.T @ Ay)
        residuals = Ay - AX @ self.beta_
        self.sigma2_ = np.sum(residuals**2) / n

        return self

    def predict(self, X_new: NDArray[np.float64],
                y_neighbors: Optional[NDArray[np.float64]] = None
                ) -> NDArray[np.float64]:
        """Predicci n considerando dependencia espacial."""
        if any(param is None for param in [self.rho_, self.beta_]):
            raise ValueError("Model not fitted")

        # Predicci n base
        base_pred = X_new @ self.beta_

        # Ajuste espacial si hay valores vecinos
        if y_neighbors is not None:
            spatial_effect = self.rho_ * (self.W @ y_neighbors)
            return base_pred + spatial_effect

        return base_pred

    def spatial_multiplier(self) -> NDArray[np.float64]:
        """Matriz multiplicador espacial (I - rho*W)^(-1)."""
```

```
        n = self.W.shape[0]
        return np.linalg.inv(np.eye(n) - self.rho_ * self.W)
```

7.2.3. Semana 7: Modelos Panel Espacial

Implementación del modelo panel espacial dinámico (13):

```
class SpatialPanelModel:
    def __init__(self, W: NDArray[np.float64],
                 spatial_lag: bool = True,
                 time_lag: bool = True) -> None:
        self.W = W
        self.spatial_lag = spatial_lag
        self.time_lag = time_lag

    def fit(self, y: NDArray[np.float64],
            X: NDArray[np.float64],
            entity_ids: NDArray[np.int64],
            time_ids: NDArray[np.int64]) -> 'SpatialPanelModel':

        # Organizar datos en panel
        y_panel, X_panel = self._organize_panel_data(
            y, X, entity_ids, time_ids)

        if self.spatial_lag and self.time_lag:
            return self._fit_spatial_temporal(y_panel, X_panel)
        elif self.spatial_lag:
            return self._fit_spatial_only(y_panel, X_panel)
        else:
            return self._fit_temporal_only(y_panel, X_panel)

    def _fit_spatial_temporal(self, y_panel: NDArray,
                              X_panel: NDArray) -> '
                                 SpatialPanelModel':
        """Estimaci n con dependencia espacial y temporal."""
        T, N = y_panel.shape

        # 2SLS con instrumentos espaciales y temporales
        instruments = self._build_instruments(y_panel, X_panel)

        # Primera etapa: regresi n de variables end genas en
           instrumentos
        endogenous_vars = self._get_endogenous_variables(y_panel)
        first_stage_fitted = self._first_stage_regression(
            endogenous_vars, instruments, X_panel)

        # Segunda etapa: regresi n principal con variables
           instrumentadas
        self._second_stage_regression(y_panel, X_panel,
           first_stage_fitted)

```

```
        return self
```

7.3. Fase 3: Machine Learning y Ensemble (Semanas 8-10)

7.3.1. Semana 8: Modelos ML Base

Implementaciones requeridas:

1. Random Forest con features espaciales
2. XGBoost con regularización temporal
3. LSTM espacial
4. Transformer con atención espacial

Random Forest espacial:

```
class SpatialRandomForest:
    def __init__(self, n_estimators: int = 100,
                 spatial_features: bool = True,
                 W: Optional[NDArray] = None) -> None:
        self.n_estimators = n_estimators
        self.spatial_features = spatial_features
        self.W = W
        self.rf_model = RandomForestRegressor(n_estimators=
           n_estimators)

    def _create_spatial_features(self, X: NDArray,
                                 y: Optional[NDArray] = None) ->
                                    NDArray:
        """Generar features espaciales adicionales."""
        if self.W is None:
            return X

        spatial_features = []

        # Lags espaciales de X
        for i in range(X.shape[1]):
            spatial_lag_x = self.W @ X[:, i]
            spatial_features.append(spatial_lag_x)

        # Lag espacial de y si disponible (para predicci n)
        if y is not None:
            spatial_lag_y = self.W @ y
            spatial_features.append(spatial_lag_y)

        # Combinar features originales y espaciales
        if spatial_features:
            spatial_array = np.column_stack(spatial_features)
            return np.column_stack([X, spatial_array])

```

```
        return X

    def fit(self, X: NDArray[np.float64],
            y: NDArray[np.float64]) -> 'SpatialRandomForest':

        if self.spatial_features:
            X_augmented = self._create_spatial_features(X, y)
        else:
            X_augmented = X

        self.rf_model.fit(X_augmented, y)
        return self

    def predict(self, X: NDArray[np.float64],
                y_neighbors: Optional[NDArray] = None) -> NDArray
                   [np.float64]:

        if self.spatial_features:
            X_augmented = self._create_spatial_features(X,
               y_neighbors)
        else:
            X_augmented = X

        return self.rf_model.predict(X_augmented)
```

7.3.2. Semana 9: Arquitectura LSTM Espacial

```
import torch
import torch.nn as nn

class SpatialTemporalLSTM(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int,
                 output_dim: int, num_layers: int = 2,
                 spatial_dim: int = None) -> None:
        super().__init__()

        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        # LSTM temporal
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers,
                            batch_first=True, dropout=0.2)

        # Capas espaciales
        if spatial_dim:
            self.spatial_conv = nn.Conv1d(hidden_dim, hidden_dim,
                                          kernel_size=3, padding=1)
            self.spatial_attention = nn.MultiheadAttention(
                hidden_dim, num_heads=8, batch_first=True)

        # Capa de salida
```

```
        self.output_layer = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x: torch.Tensor,
                spatial_adj: Optional[torch.Tensor] = None) ->
                    torch.Tensor:
        batch_size, seq_len, features = x.shape

        # LSTM temporal
        lstm_out, (hidden, cell) = self.lstm(x)

        # Procesamiento espacial si hay matriz de adyacencia
        if spatial_adj is not None and hasattr(self, '
            spatial_attention'):
            # Atenci n espacial
            spatial_out, _ = self.spatial_attention(lstm_out,
                lstm_out, lstm_out)
            lstm_out = lstm_out + spatial_out

        # Predicci n final
        output = self.dropout(lstm_out)
        output = self.output_layer(output[:, -1, :])  #  ltimo
            timestep

        return output

    def init_hidden(self, batch_size: int, device: torch.device)
        -> Tuple[torch.Tensor, torch.Tensor]:
        h0 = torch.zeros(self.num_layers, batch_size, self.
            hidden_dim).to(device)
        c0 = torch.zeros(self.num_layers, batch_size, self.
            hidden_dim).to(device)
        return h0, c0
```

7.3.3. Semana 10: Ensemble Orchestrator

Implementación del meta-learner principal:

```
class MetaEnsemble:
    def __init__(self, base_models: List[BaseModel],
                 meta_learner_type: str = 'ridge',
                 spatial_weights: Optional[NDArray] = None,
                 temporal_cv: bool = True) -> None:

        self.base_models = base_models
        self.meta_learner_type = meta_learner_type
        self.spatial_weights = spatial_weights
        self.temporal_cv = temporal_cv

        # Inicializar meta-learner
        if meta_learner_type == 'ridge':
```

```
            self.meta_learner = Ridge(alpha=1.0)
        elif meta_learner_type == 'lasso':
            self.meta_learner = Lasso(alpha=0.1)
        elif meta_learner_type == 'elastic_net':
            self.meta_learner = ElasticNet(alpha=0.1, l1_ratio
                =0.5)
        elif meta_learner_type == 'spatial_ridge':
            self.meta_learner = SpatialRidge(spatial_weights,
                alpha=1.0)
        else:
            raise ValueError(f"Unknown meta-learner: {
                meta_learner_type}")

    def fit(self, X: NDArray[np.float64],
            y: NDArray[np.float64],
            spatial_ids: Optional[NDArray] = None,
            temporal_ids: Optional[NDArray] = None) -> '
                MetaEnsemble':

        # Generar predicciones base con validaci n cruzada
        base_predictions = self._generate_base_predictions(
            X, y, spatial_ids, temporal_ids)

        # Agregar features espaciales a meta-features si
            disponible
        if self.spatial_weights is not None:
            meta_features = self._add_spatial_meta_features(
                base_predictions, spatial_ids)
        else:
            meta_features = base_predictions

        # Entrenar meta-learner
        self.meta_learner.fit(meta_features, y)

        # Entrenar modelos base en datos completos
        for model in self.base_models:
            model.fit(X, y)

        return self

    def _generate_base_predictions(self, X: NDArray, y: NDArray,
                                   spatial_ids: Optional[NDArray],
                                   temporal_ids: Optional[NDArray])
                                       -> NDArray:
        """Generar predicciones base con CV apropiada."""
        n_samples, n_models = len(X), len(self.base_models)
        base_predictions = np.zeros((n_samples, n_models))

        if self.temporal_cv and temporal_ids is not None:
            # Validaci n cruzada temporal
            cv = PurgedTimeSeriesSplit(n_splits=5, test_size=100,
```

```
                gap=10)
            cv_splits = list(cv.split(X, groups=temporal_ids))
        elif spatial_ids is not None:
            # Validaci n cruzada espacial
            cv = SpatialBlockCV(n_splits=5)
            cv_splits = list(cv.split(X, groups=spatial_ids))
        else:
            # Validaci n cruzada est ndar (solo si no hay estructura)
            cv = KFold(n_splits=5, shuffle=True, random_state=42)
            cv_splits = list(cv.split(X))

        for train_idx, val_idx in cv_splits:
            X_train, X_val = X[train_idx], X[val_idx]
            y_train = y[train_idx]

            for model_idx, model in enumerate(self.base_models):
                model_copy = clone(model)
                model_copy.fit(X_train, y_train)
                predictions = model_copy.predict(X_val)
                base_predictions[val_idx, model_idx] = predictions

        return base_predictions

    def predict(self, X: NDArray[np.float64],
                spatial_context: Optional[NDArray] = None) -> NDArray[np.float64]:
        """Generar predicci n ensemble."""

        # Predicciones de modelos base
        base_predictions = np.zeros((len(X), len(self.base_models)))
        for i, model in enumerate(self.base_models):
            base_predictions[:, i] = model.predict(X)

        # Agregar features espaciales si disponible
        if self.spatial_weights is not None and spatial_context is not None:
            meta_features = self._add_spatial_meta_features(
                base_predictions, spatial_context)
        else:
            meta_features = base_predictions

        # Predicci n final del meta-learner
        return self.meta_learner.predict(meta_features)

    def get_model_importances(self) -> Dict[str, float]:
        """Obtener importancia de cada modelo base."""
        if hasattr(self.meta_learner, 'coef_'):
            coefs = np.abs(self.meta_learner.coef_)
```

```
            n_base_models = len(self.base_models)
            base_coefs = coefs[:n_base_models]  # Primeros coefs son de modelos base

            importances = {}
            for i, model in enumerate(self.base_models):
                model_name = model.__class__.__name__
                importances[model_name] = base_coefs[i]

            return importances
        else:
            return {model.__class__.__name__: 1.0/len(self.base_models)
                    for model in self.base_models}
```

8. Métricas de Evaluación Avanzadas

8.1. Métricas Específicas Espacio-Temporales

8.1.1. Coherencia Espacial Generalizada

Extendiendo (17), definimos:

$$\text{GSC}_t(\alpha) = 1 - \frac{\sum_{i,j} w_{ij} |\varepsilon_{i,t} - \varepsilon_{j,t}|^\alpha}{\sum_{i,j} w_{ij} (|\varepsilon_{i,t}|^\alpha + |\varepsilon_{j,t}|^\alpha)} \tag{25}$$

donde $\alpha \in [1, 2]$ controla la sensibilidad a outliers.

8.1.2. Persistencia Temporal de Errores

$$\text{TPE}_i = \frac{1}{H} \sum_{h=1}^{H} \text{corr}(\varepsilon_{i,t}, \varepsilon_{i,t+h}) \tag{26}$$

8.2. Tests de Superior Predictive Ability

Implementación del test de White (2000) para comparación múltiple:

```
def reality_check_test(losses: NDArray[np.float64],
                       benchmark_losses: NDArray[np.float64],
                       bootstrap_samples: int = 10000) -> Tuple[float, float]:
    """
    Test de Reality Check para comparaci n m ltiple de modelos.

    Parameters
    ----------
    losses : NDArray, shape (n_observations, n_models)
        P rdidas de cada modelo
```

```
    benchmark_losses : NDArray, shape (n_observations,)
        P rdidas del modelo benchmark
    bootstrap_samples : int
        N mero de muestras bootstrap

    Returns
    -------
    test_stat : float
        Estad stico del test
    p_value : float
        P-valor bootstrap
    """
    n_obs, n_models = losses.shape

    # Diferencias de p rdidas vs benchmark
    loss_diffs = losses - benchmark_losses.reshape(-1, 1)
    mean_diffs = np.mean(loss_diffs, axis=0)

    # Estad stico del test (m ximo de las diferencias)
    test_stat = np.max(mean_diffs)

    # Bootstrap para distribuci n bajo H0
    bootstrap_stats = []

    for _ in range(bootstrap_samples):
        # Recentrar diferencias bajo H0
        centered_diffs = loss_diffs - mean_diffs

        # Muestra bootstrap
        bootstrap_indices = np.random.choice(n_obs, n_obs,
           replace=True)
        bootstrap_diffs = centered_diffs[bootstrap_indices]

        # Estad stico bootstrap
        bootstrap_mean = np.mean(bootstrap_diffs, axis=0)
        bootstrap_stat = np.max(bootstrap_mean)
        bootstrap_stats.append(bootstrap_stat)

    # P-valor como proporci n de estad sticos bootstrap >=
       test_stat
    p_value = np.mean(np.array(bootstrap_stats) >= test_stat)

    return test_stat, p_value


def model_confidence_set(losses: NDArray[np.float64],
                         confidence_level: float = 0.95,
                         bootstrap_samples: int = 5000) -> List[
                            int]:
    """
    Model Confidence Set de Hansen, Lunde y Nason (2011).
```

```
59
60      Returns
61      -------
62      mcs_models : List[int]
63          ndices de modelos en el MCS
64      """
65      n_obs, n_models = losses.shape
66      active_models = list(range(n_models))
67
68      while len(active_models) > 1:
69          # Calcular estad sticos t para modelos activos
70          active_losses = losses[:, active_models]
71          t_stats = []
72
73          for i in range(len(active_models)):
74              for j in range(i + 1, len(active_models)):
75                  loss_diff = active_losses[:, i] - active_losses
                        [:, j]
76                  t_stat = np.mean(loss_diff) / (np.std(loss_diff)
                        / np.sqrt(n_obs))
77                  t_stats.append((abs(t_stat), active_models[i],
                        active_models[j]))
78
79          # Encontrar el par con mayor estad stico t
80          max_t_stat, model_i, model_j = max(t_stats)
81
82          # Bootstrap test
83          loss_diff_ij = losses[:, model_i] - losses[:, model_j]
84          centered_diff = loss_diff_ij - np.mean(loss_diff_ij)
85
86          bootstrap_t_stats = []
87          for _ in range(bootstrap_samples):
88              bootstrap_sample = np.random.choice(centered_diff,
                    n_obs, replace=True)
89              bootstrap_t = np.mean(bootstrap_sample) / (np.std(
                    bootstrap_sample) / np.sqrt(n_obs))
90              bootstrap_t_stats.append(abs(bootstrap_t))
91
92          p_value = np.mean(np.array(bootstrap_t_stats) >=
                max_t_stat)
93
94          # Eliminar modelo con peor performance si significativo
95          if p_value < (1 - confidence_level):
96              mean_loss_i = np.mean(losses[:, model_i])
97              mean_loss_j = np.mean(losses[:, model_j])
98
99              if mean_loss_i > mean_loss_j:
100                 active_models.remove(model_i)
101             else:
102                 active_models.remove(model_j)
103         else:
```

```
            break  # No hay diferencias significativas

    return active_models
```

9. Optimización Avanzada y Regularización

9.1. Optimización Bayesiana de Hiperparámetros

```
import optuna
from optuna.samplers import TPESampler
from optuna.pruners import HyperbandPruner

class BayesianEnsembleOptimizer:
    def __init__(self, base_models: List[BaseModel],
                 n_trials: int = 200,
                 cv_folds: int = 5) -> None:
        self.base_models = base_models
        self.n_trials = n_trials
        self.cv_folds = cv_folds

        # Configurar Optuna
        self.sampler = TPESampler(seed=42)
        self.pruner = HyperbandPruner(min_resource=1,
           max_resource=cv_folds)

    def objective(self, trial: optuna.Trial,
                  X: NDArray[np.float64],
                  y: NDArray[np.float64]) -> float:
        """Funci n objetivo para optimizaci n bayesiana."""

        # Optimizar hiperpar metros de cada modelo base
        optimized_models = []

        for i, model in enumerate(self.base_models):
            model_name = f"model_{i}_{model.__class__.__name__}"

            if isinstance(model, RandomForestRegressor):
                n_estimators = trial.suggest_int(f"{model_name}
                   _n_estimators", 50, 500)
                max_depth = trial.suggest_int(f"{model_name}
                   _max_depth", 3, 20)
                min_samples_split = trial.suggest_int(f"{
                   model_name}_min_samples_split", 2, 20)

                optimized_model = RandomForestRegressor(
                    n_estimators=n_estimators,
                    max_depth=max_depth,
                    min_samples_split=min_samples_split,
                    random_state=42
                )
```

```
39
40              elif isinstance(model, XGBRegressor):
41                  n_estimators = trial.suggest_int(f"{model_name}
                        _n_estimators", 50, 1000)
42                  learning_rate = trial.suggest_float(f"{model_name
                        }_learning_rate", 0.01, 0.3)
43                  max_depth = trial.suggest_int(f"{model_name}
                        _max_depth", 3, 10)
44                  subsample = trial.suggest_float(f"{model_name}
                        _subsample", 0.6, 1.0)
45
46                  optimized_model = XGBRegressor(
47                      n_estimators=n_estimators,
48                      learning_rate=learning_rate,
49                      max_depth=max_depth,
50                      subsample=subsample,
51                      random_state=42
52                  )
53
54              else:
55                  optimized_model = model  # Usar configuraci n
                        por defecto
56
57              optimized_models.append(optimized_model)
58
59          # Optimizar pesos del ensemble
60          ensemble_method = trial.suggest_categorical("
                ensemble_method",
61                                                      ["
                                                            simple_average
                                                            ", "
                                                            weighted_average
                                                            ", "stacking
                                                            "])
62
63          if ensemble_method == "weighted_average":
64              # Optimizar pesos directamente
65              raw_weights = []
66              for i in range(len(optimized_models)):
67                  weight = trial.suggest_float(f"weight_{i}", 0.0,
                        1.0)
68                  raw_weights.append(weight)
69
70              # Normalizar pesos
71              weight_sum = sum(raw_weights)
72              if weight_sum == 0:
73                  weights = [1.0 / len(optimized_models)] * len(
                        optimized_models)
74              else:
75                  weights = [w / weight_sum for w in raw_weights]
76
```

```
        # Evaluar ensemble con validaci n cruzada
        cv_scores = []
        tscv = PurgedTimeSeriesSplit(n_splits=self.cv_folds,
            test_size=100, gap=5)

        for fold, (train_idx, val_idx) in enumerate(tscv.split(X)
            ):
            X_train, X_val = X[train_idx], X[val_idx]
            y_train, y_val = y[train_idx], y[val_idx]

            # Entrenar modelos base
            fold_predictions = []
            for model in optimized_models:
                model_copy = clone(model)
                model_copy.fit(X_train, y_train)
                pred = model_copy.predict(X_val)
                fold_predictions.append(pred)

            # Combinar predicciones seg n m todo
            if ensemble_method == "simple_average":
                ensemble_pred = np.mean(fold_predictions, axis=0)
            elif ensemble_method == "weighted_average":
                ensemble_pred = np.average(fold_predictions, axis
                    =0, weights=weights)
            elif ensemble_method == "stacking":
                # Meta-learner simple para stacking
                meta_X = np.column_stack(fold_predictions)
                meta_learner = Ridge(alpha=1.0)

                # Usar parte del validation set para entrenar
                    meta-learner
                if len(val_idx) > 50:
                    meta_train_size = len(val_idx) // 2
                    meta_train_idx = val_idx[:meta_train_size]
                    meta_val_idx = val_idx[meta_train_size:]

                    # Re-predecir para meta-training set
                    meta_train_preds = []
                    for model in optimized_models:
                        model_copy = clone(model)
                        model_copy.fit(X_train, y_train)
                        meta_pred = model_copy.predict(X[
                            meta_train_idx])
                        meta_train_preds.append(meta_pred)

                    meta_X_train = np.column_stack(
                        meta_train_preds)
                    meta_learner.fit(meta_X_train, y[
                        meta_train_idx])

                    # Predecir en meta-validation set
```

```
                    meta_X_val = np.column_stack([pred[len(
                        meta_train_idx):]
                                                  for pred in
                                                      fold_predictions
                                                      ])
                    ensemble_pred = meta_learner.predict(
                        meta_X_val)
                    y_val = y[meta_val_idx]
                else:
                    # Fallback a simple average si validation set
                        muy peque o
                    ensemble_pred = np.mean(fold_predictions,
                        axis=0)

            # Calcular error
            rmse = np.sqrt(np.mean((y_val - ensemble_pred) ** 2))
            cv_scores.append(rmse)

            # Pruning para optimizaci n eficiente
            trial.report(rmse, fold)
            if trial.should_prune():
                raise optuna.TrialPruned()

        return np.mean(cv_scores)

    def optimize(self, X: NDArray[np.float64],
                 y: NDArray[np.float64]) -> Dict[str, Any]:
        """Ejecutar optimizaci n bayesiana."""

        study = optuna.create_study(
            direction="minimize",
            sampler=self.sampler,
            pruner=self.pruner
        )

        # Ejecutar optimizaci n
        study.optimize(
            lambda trial: self.objective(trial, X, y),
            n_trials=self.n_trials,
            show_progress_bar=True
        )

        return {
            "best_params": study.best_params,
            "best_value": study.best_value,
            "n_trials": len(study.trials),
            "study": study
        }
```

9.2. Regularización Espacial

Para el meta-learner con regularización espacial:

$$\min_{\boldsymbol{\beta}} \frac{1}{2N}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda_1\|\boldsymbol{\beta}\|_1 + \lambda_2\boldsymbol{\beta}^T\mathbf{L}\boldsymbol{\beta} \tag{27}$$

donde $\mathbf{L} = \mathbf{D} - \mathbf{W}$ es la matriz Laplaciana espacial.

```python
class SpatialRidge:
    def __init__(self, spatial_weights: NDArray[np.float64],
                 alpha: float = 1.0,
                 spatial_alpha: float = 1.0) -> None:
        self.W = spatial_weights
        self.alpha = alpha
        self.spatial_alpha = spatial_alpha
        self.coef_: Optional[NDArray] = None
        self.intercept_: Optional[float] = None

    def fit(self, X: NDArray[np.float64],
            y: NDArray[np.float64]) -> 'SpatialRidge':
        """Ajuste con regularizaci n Ridge + espacial."""

        n_samples, n_features = X.shape

        # Matriz Laplaciana espacial
        D = np.diag(np.sum(self.W, axis=1))
        L = D - self.W

        # Centrar datos
        X_centered = X - np.mean(X, axis=0)
        y_centered = y - np.mean(y)

        # Matriz de regularizaci n combinada
        I = np.eye(n_features)
        regularization_matrix = self.alpha * I

        # Agregar regularizaci n espacial si las dimensiones
           coinciden
        if L.shape[0] == n_features:
            regularization_matrix += self.spatial_alpha * L

        # Soluci n anal tica
        XtX = X_centered.T @ X_centered
        XtX_reg = XtX + n_samples * regularization_matrix

        try:
            self.coef_ = np.linalg.solve(XtX_reg, X_centered.T @
               y_centered)
        except np.linalg.LinAlgError:
            # Fallback a pseudo-inversa si singular
            XtX_reg += 1e-6 * np.eye(n_features)
```

```
            self.coef_ = np.linalg.solve(XtX_reg, X_centered.T @
                y_centered)

        self.intercept_ = np.mean(y) - np.mean(X, axis=0) @ self.
            coef_

        return self

    def predict(self, X: NDArray[np.float64]) -> NDArray[np.
        float64]:
        if self.coef_ is None:
            raise ValueError("Model not fitted")

        return X @ self.coef_ + self.intercept_
```

10. Monitoring y Drift Detection

10.1. Detección de Drift Multivariado

```
from scipy import stats
from sklearn.metrics import roc_auc_score

class MultivariateDriftDetector:
    def __init__(self, window_size: int = 1000,
                 significance_level: float = 0.05,
                 drift_threshold: float = 0.7) -> None:
        self.window_size = window_size
        self.alpha = significance_level
        self.drift_threshold = drift_threshold
        self.reference_data: Optional[NDArray] = None
        self.reference_stats: Optional[Dict] = None

    def set_reference(self, X_reference: NDArray[np.float64]) ->
        None:
        """Establecer datos de referencia."""
        self.reference_data = X_reference
        self.reference_stats = {
            'mean': np.mean(X_reference, axis=0),
            'cov': np.cov(X_reference.T),
            'n_samples': len(X_reference)
        }

    def detect_covariate_drift(self, X_current: NDArray[np.
        float64]) -> Dict[str, float]:
        """Detectar drift en covariables usando m ltiples
            m todos."""
        if self.reference_data is None:
            raise ValueError("Reference data not set")

        results = {}
```

```
29
30          # 1. Test de Kolmogorov-Smirnov por feature
31          ks_pvalues = []
32          for i in range(X_current.shape[1]):
33              ks_stat, ks_pval = stats.ks_2samp(
34                  self.reference_data[:, i],
35                  X_current[:, i]
36              )
37              ks_pvalues.append(ks_pval)
38
39          results['ks_min_pvalue'] = np.min(ks_pvalues)
40          results['ks_drift_detected'] = results['ks_min_pvalue'] <
                self.alpha
41
42          # 2. Test de m xima diferencia de medias (Hotelling T )
43          if X_current.shape[1] > 1:
44              mean_diff = np.mean(X_current, axis=0) - self.
                   reference_stats['mean']
45
46              # Pooled covariance
47              n1, n2 = len(self.reference_data), len(X_current)
48              S1 = np.cov(self.reference_data.T)
49              S2 = np.cov(X_current.T)
50              S_pooled = ((n1-1)*S1 + (n2-1)*S2) / (n1 + n2 - 2)
51
52              # Hotelling T  statistic
53              try:
54                  T2 = (n1 * n2) / (n1 + n2) * mean_diff.T @ np.
                       linalg.inv(S_pooled) @ mean_diff
55
56                  # F-distribution approximation
57                  p = X_current.shape[1]
58                  F_stat = T2 * (n1 + n2 - p - 1) / ((n1 + n2 - 2)
                       * p)
59                  hotelling_pval = 1 - stats.f.cdf(F_stat, p, n1 +
                       n2 - p - 1)
60
61                  results['hotelling_t2'] = T2
62                  results['hotelling_pvalue'] = hotelling_pval
63                  results['hotelling_drift'] = hotelling_pval <
                       self.alpha
64              except np.linalg.LinAlgError:
65                  results['hotelling_t2'] = np.nan
66                  results['hotelling_pvalue'] = np.nan
67                  results['hotelling_drift'] = False
68
69          # 3. Discriminator-based drift detection
70          auc_score = self._discriminator_drift_test(X_current)
71          results['discriminator_auc'] = auc_score
72          results['discriminator_drift'] = auc_score > self.
               drift_threshold
```

```
73
74          # 4. Decisi n final
75          drift_indicators = [
76              results.get('ks_drift_detected', False),
77              results.get('hotelling_drift', False),
78              results.get('discriminator_drift', False)
79          ]
80          results['overall_drift_detected'] = any(drift_indicators)
81
82          return results
83
84      def _discriminator_drift_test(self, X_current: NDArray[np.
        float64]) -> float:
85          """Test de drift usando clasificador discriminativo."""
86          from sklearn.ensemble import RandomForestClassifier
87          from sklearn.model_selection import cross_val_score
88
89          # Crear dataset combinado con labels
90          X_combined = np.vstack([self.reference_data, X_current])
91          y_combined = np.hstack([
92              np.zeros(len(self.reference_data)),  # Reference = 0
93              np.ones(len(X_current))              # Current = 1
94          ])
95
96          # Entrenar clasificador
97          clf = RandomForestClassifier(n_estimators=100,
            random_state=42)
98
99          # Cross-validation AUC
100         auc_scores = cross_val_score(clf, X_combined, y_combined,
101                                      cv=5, scoring='roc_auc')
102
103         return np.mean(auc_scores)
104
105     def detect_prediction_drift(self,
106                                 errors_reference: NDArray[np.
                                      float64],
107                                 errors_current: NDArray[np.float64
                                      ]) -> Dict[str, float]:
108         """Detectar drift en errores de predicci n."""
109
110         results = {}
111
112         # 1. Test de cambio en media de errores
113         t_stat, t_pval = stats.ttest_ind(errors_reference,
            errors_current)
114         results['mean_change_tstat'] = t_stat
115         results['mean_change_pvalue'] = t_pval
116         results['mean_change_detected'] = t_pval < self.alpha
117
118         # 2. Test de cambio en varianza
```

```
        f_stat = np.var(errors_current) / np.var(errors_reference)
        f_pval = 2 * min(
            stats.f.cdf(f_stat, len(errors_current)-1, len(errors_reference)-1),
            1 - stats.f.cdf(f_stat, len(errors_current)-1, len(errors_reference)-1)
        )
        results['variance_change_fstat'] = f_stat
        results['variance_change_pvalue'] = f_pval
        results['variance_change_detected'] = f_pval < self.alpha

        # 3. Test de Kolmogorov-Smirnov en distribuci n de errores
        ks_stat, ks_pval = stats.ks_2samp(errors_reference, errors_current)
        results['distribution_change_ks'] = ks_stat
        results['distribution_change_pvalue'] = ks_pval
        results['distribution_change_detected'] = ks_pval < self.alpha

        # Decisi n final
        change_indicators = [
            results['mean_change_detected'],
            results['variance_change_detected'],
            results['distribution_change_detected']
        ]
        results['overall_change_detected'] = any(change_indicators)

        return results
```

10.2. Sistema de Alertas y Reentrenamiento

```
class ModelMonitoringSystem:
    def __init__(self, ensemble_model: MetaEnsemble,
                 drift_detector: MultivariateDriftDetector,
                 retraining_threshold: float = 0.15,
                 performance_window: int = 100) -> None:

        self.ensemble = ensemble_model
        self.drift_detector = drift_detector
        self.retraining_threshold = retraining_threshold
        self.performance_window = performance_window

        # M tricas de monitoreo
        self.performance_history = []
        self.drift_history = []
        self.last_retrain_time = None

```

```python
    def monitor_prediction_batch(self,
                                 X: NDArray[np.float64],
                                 y_true: NDArray[np.float64],
                                 timestamp: pd.Timestamp) -> Dict[
                                    str, Any]:
        """Monitorear batch de predicciones."""

        # Generar predicciones
        y_pred = self.ensemble.predict(X)

        # Calcular m tricas de performance
        rmse = np.sqrt(np.mean((y_true - y_pred)**2))
        mae = np.mean(np.abs(y_true - y_pred))

        # Detectar drift en covariables
        drift_results = self.drift_detector.
            detect_covariate_drift(X)

        # Detectar drift en errores si hay historial suficiente
        errors = y_true - y_pred
        if len(self.performance_history) >= self.
            performance_window:
            reference_errors = np.concatenate([
                batch['errors'] for batch in self.
                    performance_history[-self.performance_window:]
            ])
            error_drift = self.drift_detector.
                detect_prediction_drift(
                reference_errors, errors)
        else:
            error_drift = {'overall_change_detected': False}

        # Guardar m tricas
        batch_metrics = {
            'timestamp': timestamp,
            'rmse': rmse,
            'mae': mae,
            'n_samples': len(X),
            'errors': errors,
            'covariate_drift': drift_results,
            'error_drift': error_drift
        }

        self.performance_history.append(batch_metrics)

        # Determinar si se necesita reentrenamiento
        retraining_needed = self._evaluate_retraining_need(
            batch_metrics)

        # Alertas
```

```
61          alerts = self._generate_alerts(batch_metrics,
                retraining_needed)
62
63          return {
64              'performance_metrics': {
65                  'rmse': rmse,
66                  'mae': mae,
67                  'n_predictions': len(X)
68              },
69              'drift_detection': drift_results,
70              'error_drift': error_drift,
71              'retraining_recommended': retraining_needed,
72              'alerts': alerts,
73              'timestamp': timestamp
74          }
75
76      def _evaluate_retraining_need(self, current_metrics: Dict) ->
            bool:
77          """Evaluar si se necesita reentrenamiento."""
78
79          # 1. Drift significativo detectado
80          if (current_metrics['covariate_drift']['
                overall_drift_detected'] or
81              current_metrics['error_drift']['
                    overall_change_detected']):
82              return True
83
84          # 2. Degradaci n sostenida de performance
85          if len(self.performance_history) >= 10:
86              recent_rmse = [batch['rmse'] for batch in self.
                    performance_history[-10:]]
87              baseline_rmse = [batch['rmse'] for batch in self.
                    performance_history[-50:-10]]
88
89              if len(baseline_rmse) > 0:
90                  relative_degradation = (np.mean(recent_rmse) - np
                        .mean(baseline_rmse)) / np.mean(baseline_rmse)
91                  if relative_degradation > self.
                        retraining_threshold:
92                      return True
93
94          # 3. Tiempo desde  ltimo  reentrenamiento
95          if self.last_retrain_time is not None:
96              days_since_retrain = (current_metrics['timestamp'] -
                    self.last_retrain_time).days
97              if days_since_retrain > 90:  # Reentrenamiento
                    trimestral m nimo
98                  return True
99
100         return False
101
```

```
102     def _generate_alerts(self, metrics: Dict, retraining_needed:
            bool) -> List[Dict]:
103         """Generar alertas basadas en m tricas."""
104         alerts = []
105 
106         # Alert por performance degradada
107         if len(self.performance_history) >= 5:
108             recent_rmse = np.mean([batch['rmse'] for batch in
                    self.performance_history[-5:]])
109             if len(self.performance_history) >= 20:
110                 baseline_rmse = np.mean([batch['rmse'] for batch
                        in self.performance_history[-20:-5]])
111                 if recent_rmse > baseline_rmse * 1.2:
112                     alerts.append({
113                         'type': 'performance_degradation',
114                         'severity': 'warning',
115                         'message': f'RMSE␣aument ␣{(recent_rmse/
                                baseline_rmse␣-␣1)*100:.1f}%␣vs␣
                                baseline',
116                         'metric': 'rmse',
117                         'current_value': recent_rmse,
118                         'baseline_value': baseline_rmse
119                     })
120 
121         # Alert por drift
122         if metrics['covariate_drift']['overall_drift_detected']:
123             alerts.append({
124                 'type': 'covariate_drift',
125                 'severity': 'warning',
126                 'message': 'Drift␣detectado␣en␣variables␣de␣
                        entrada',
127                 'details': metrics['covariate_drift']
128             })
129 
130         if metrics['error_drift']['overall_change_detected']:
131             alerts.append({
132                 'type': 'prediction_drift',
133                 'severity': 'warning',
134                 'message': 'Drift␣detectado␣en␣distribuci n␣de␣
                        errores',
135                 'details': metrics['error_drift']
136             })
137 
138         # Alert cr tica si reentrenamiento necesario
139         if retraining_needed:
140             alerts.append({
141                 'type': 'retraining_required',
142                 'severity': 'critical',
143                 'message': 'Se␣recomienda␣reentrenamiento␣del␣
                        modelo',
144                 'timestamp': metrics['timestamp']
```

```
            })

        return alerts

    def retrain_model(self, X_new: NDArray[np.float64],
                      y_new: NDArray[np.float64],
                      timestamp: pd.Timestamp) -> Dict[str, Any]:
        """Reentrenar el modelo ensemble."""

        # Backup del modelo actual
        model_backup = deepcopy(self.ensemble)

        try:
            # Reentrenamiento
            self.ensemble.fit(X_new, y_new)

            # Actualizar timestamp
            self.last_retrain_time = timestamp

            # Validar que el nuevo modelo es mejor
            validation_results = self._validate_retrained_model(
                model_backup, X_new, y_new)

            if not validation_results['improvement_significant']:
                # Rollback si no hay mejora
                self.ensemble = model_backup
                return {
                    'success': False,
                    'reason': 'No significant improvement',
                    'validation_results': validation_results
                }

            return {
                'success': True,
                'timestamp': timestamp,
                'validation_results': validation_results,
                'new_model_performance': validation_results['
                   new_model_metrics']
            }

        except Exception as e:
            # Rollback en caso de error
            self.ensemble = model_backup
            return {
                'success': False,
                'reason': f'Training failed: {str(e)}',
                'error': str(e)
            }

    def _validate_retrained_model(self, old_model: MetaEnsemble,
                                  X_val: NDArray[np.float64],
```

```
                                    y_val: NDArray[np.float64]) ->
                                       Dict[str, Any]:
        """Validar modelo reentrenado vs modelo anterior."""

        # Predicciones de ambos modelos
        y_pred_old = old_model.predict(X_val)
        y_pred_new = self.ensemble.predict(X_val)

        # M tricas de performance
        rmse_old = np.sqrt(np.mean((y_val - y_pred_old)**2))
        rmse_new = np.sqrt(np.mean((y_val - y_pred_new)**2))

        mae_old = np.mean(np.abs(y_val - y_pred_old))
        mae_new = np.mean(np.abs(y_val - y_pred_new))

        # Test de significancia estad stica
        errors_old = y_val - y_pred_old
        errors_new = y_val - y_pred_new

        dm_stat, dm_pval = diebold_mariano_test(
            errors_old**2, errors_new**2, horizon=1)

        improvement_significant = (dm_pval < 0.05) and (rmse_new
           < rmse_old)

        return {
            'old_model_metrics': {'rmse': rmse_old, 'mae':
               mae_old},
            'new_model_metrics': {'rmse': rmse_new, 'mae':
               mae_new},
            'improvement_pct': (rmse_old - rmse_new) / rmse_old *
                100,
            'dm_test_stat': dm_stat,
            'dm_test_pvalue': dm_pval,
            'improvement_significant': improvement_significant
        }
```

11. Criterios de Éxito y Benchmarking

11.1. KPIs Técnicos Obligatorios

Calidad de Código:

$$\text{Code Coverage} \geq 95\,\% \tag{28}$$

$$\text{Cyclomatic Complexity} < 10 \text{ por función} \tag{29}$$

$$\text{Type Annotation Coverage} = 100\,\% \text{ API pública} \tag{30}$$

Performance Computacional:

$$\text{Latencia Predicción} < 100\text{ms para } N = 1000 \text{ regiones} \tag{31}$$

$$\text{Throughput Entrenamiento} > 10^4 \text{ obs/segundo} \tag{32}$$

$$\text{Memory Usage} < 8\text{GB para } 10^6 \text{ observaciones} \tag{33}$$

Accuracy Benchmarks:

$$\text{RMSE Improvement vs Naive} > 20\,\% \tag{34}$$

$$\text{Directional Accuracy} > 60\,\% \tag{35}$$

$$\text{Spatial Coherence Index} > 0{,}8 \tag{36}$$

11.2. Tests de Aceptación

```python
class AcceptanceTests:
    """Tests de aceptaci n para validar criterios de  xito ."""

    def __init__(self, ensemble_model: MetaEnsemble,
                 baseline_models: Dict[str, BaseModel],
                 test_data: Dict[str, NDArray]) -> None:
        self.ensemble = ensemble_model
        self.baselines = baseline_models
        self.test_data = test_data

    def test_accuracy_requirements(self) -> Dict[str, bool]:
        """Validar requisitos de accuracy vs baselines."""

        X_test = self.test_data['X']
        y_test = self.test_data['y']

        # Predicciones ensemble
        y_pred_ensemble = self.ensemble.predict(X_test)
        rmse_ensemble = np.sqrt(np.mean((y_test - y_pred_ensemble
            )**2))

        results = {}

        # Test vs Naive forecasts
        naive_forecast = np.full_like(y_test, np.mean(y_test))
        rmse_naive = np.sqrt(np.mean((y_test - naive_forecast)
            **2))

        rmse_improvement = (rmse_naive - rmse_ensemble) /
            rmse_naive
        results['rmse_vs_naive'] = rmse_improvement > 0.20

        # Directional accuracy
        if len(y_test) > 1:
            actual_direction = np.diff(y_test) > 0
            pred_direction = np.diff(y_pred_ensemble) > 0
```

```
            da = np.mean(actual_direction == pred_direction)
            results['directional_accuracy'] = da > 0.60
        else:
            results['directional_accuracy'] = True

        # Test vs otros baselines
        for name, baseline in self.baselines.items():
            try:
                y_pred_baseline = baseline.predict(X_test)
                rmse_baseline = np.sqrt(np.mean((y_test -
                    y_pred_baseline)**2))

                improvement = (rmse_baseline - rmse_ensemble) /
                    rmse_baseline
                results[f'rmse_vs_{name}'] = improvement > 0.05

                # Test estad stico de significancia
                errors_ensemble = (y_test - y_pred_ensemble)**2
                errors_baseline = (y_test - y_pred_baseline)**2

                dm_stat, dm_pval = diebold_mariano_test(
                    errors_baseline, errors_ensemble, horizon=1)
                results[f'significant_vs_{name}'] = dm_pval <
                    0.05 and improvement > 0

            except Exception as e:
                results[f'error_{name}'] = str(e)

        return results

    def test_spatial_coherence(self, W: NDArray[np.float64]) ->
        Dict[str, float]:
        """Test de coherencia espacial."""

        X_test = self.test_data['X']
        y_test = self.test_data['y']
        y_pred = self.ensemble.predict(X_test)

        errors = y_test - y_pred

        # Calcular coherencia espacial
        numerator = 0
        denominator = 0

        for i in range(len(errors)):
            for j in range(len(errors)):
                if W[i, j] > 0:
                    numerator += W[i, j] * abs(errors[i] - errors
                        [j])
                    denominator += W[i, j] * (abs(errors[i]) +
                        abs(errors[j]))
```

```
79
80          if denominator > 0:
81              spatial_coherence = 1 - (numerator / denominator)
82          else:
83              spatial_coherence = 1.0
84
85          return {
86              'spatial_coherence_index': spatial_coherence,
87              'meets_requirement': spatial_coherence > 0.8
88          }
89
90      def test_performance_requirements(self) -> Dict[str, bool]:
91          """Test de requirements de performance."""
92          import time
93          import psutil
94          import os
95
96          results = {}
97
98          # Test de latencia de predicci n
99          X_test_1k = self.test_data['X'][:1000]  # 1000 regiones
100
101         start_time = time.time()
102         _ = self.ensemble.predict(X_test_1k)
103         prediction_time = (time.time() - start_time) * 1000  # ms
104
105         results['latency_requirement'] = prediction_time < 100.0
106         results['actual_latency_ms'] = prediction_time
107
108         # Test de memory usage durante predicci n
109         process = psutil.Process(os.getpid())
110         memory_before = process.memory_info().rss / 1024 / 1024
               # MB
111
112         # Predicci n en dataset grande
113         if len(self.test_data['X']) >= 100000:
114             X_large = self.test_data['X'][:100000]
115             _ = self.ensemble.predict(X_large)
116
117             memory_after = process.memory_info().rss / 1024 /
                   1024  # MB
118             memory_used = memory_after - memory_before
119
120             results['memory_requirement'] = memory_used < 1000  #
                    < 1GB
121             results['actual_memory_mb'] = memory_used
122         else:
123             results['memory_requirement'] = True
124             results['actual_memory_mb'] = 0
125
126         return results
```

```
127
128     def test_model_robustness(self) -> Dict[str, bool]:
129         """Tests de robustez del modelo."""
130
131         results = {}
132         X_test = self.test_data['X']
133         y_test = self.test_data['y']
134
135         # Test con outliers artificiales
136         X_outliers = X_test.copy()
137         outlier_indices = np.random.choice(len(X_outliers), size
                =10, replace=False)
138         X_outliers[outlier_indices] *= 5  # Outliers extremos
139
140         try:
141             y_pred_outliers = self.ensemble.predict(X_outliers)
142             y_pred_normal = self.ensemble.predict(X_test)
143
144             # El modelo debe ser relativamente robusto
145             prediction_change = np.mean(np.abs(y_pred_outliers -
                    y_pred_normal))
146             baseline_std = np.std(y_pred_normal)
147
148             results['outlier_robustness'] = prediction_change < 2
                    * baseline_std
149
150         except Exception:
151             results['outlier_robustness'] = False
152
153         # Test con datos faltantes
154         X_missing = X_test.copy()
155         missing_mask = np.random.random(X_missing.shape) < 0.05
                # 5% missing
156         X_missing[missing_mask] = np.nan
157
158         try:
159             # El modelo debe manejar NaNs o fallar graciosamente
160             y_pred_missing = self.ensemble.predict(X_missing)
161             results['missing_data_handling'] = not np.any(np.
                    isnan(y_pred_missing))
162
163         except Exception:
164             # Si falla, debe ser por validaci n adecuada, no por
                    error inesperado
165             results['missing_data_handling'] = True
166
167         return results
168
169     def generate_acceptance_report(self) -> Dict[str, Any]:
170         """Generar reporte completo de aceptaci n."""
171
```

```
        report = {
            'timestamp': pd.Timestamp.now(),
            'test_results': {},
            'overall_pass': True,
            'failed_tests': []
        }

        # Ejecutar todos los tests
        accuracy_results = self.test_accuracy_requirements()
        performance_results = self.test_performance_requirements
            ()
        robustness_results = self.test_model_robustness()

        # Espacial coherence test si hay matriz W disponible
        if 'W' in self.test_data:
            spatial_results = self.test_spatial_coherence(self.
                test_data['W'])
        else:
            spatial_results = {'meets_requirement': True}

        # Consolidar resultados
        all_results = {
            **accuracy_results,
            **performance_results,
            **robustness_results,
            **spatial_results
        }

        report['test_results'] = all_results

        # Identificar tests fallidos
        for test_name, passed in all_results.items():
            if isinstance(passed, bool) and not passed:
                report['failed_tests'].append(test_name)
                report['overall_pass'] = False

        # Calcular score de aceptaci n
        boolean_results = {k: v for k, v in all_results.items()
                           if isinstance(v, bool)}

        if boolean_results:
            acceptance_score = sum(boolean_results.values()) /
                len(boolean_results)
            report['acceptance_score'] = acceptance_score
            report['minimum_score_met'] = acceptance_score >=
                0.85
        else:
            report['acceptance_score'] = 0.0
            report['minimum_score_met'] = False

        return report
```

12. Deployment y Productización

12.1. Containerización

Dockerfile de producción:

```
FROM python:3.11-slim as builder

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y \
    build-essential \
    gfortran \
    libopenblas-dev \
    liblapack-dev \
    && rm -rf /var/lib/apt/lists/*

# Crear usuario no-root
RUN useradd --create-home --shell /bin/bash ensemble

WORKDIR /app

# Instalar dependencias Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar c digo fuente
COPY --chown=ensemble:ensemble . .

# Instalar paquete
RUN pip install -e .

# Ejecutar tests
RUN python -m pytest tests/ -v --cov=src --cov-report=term-
   missing

# Stage de producci n
FROM python:3.11-slim as production

RUN apt-get update && apt-get install -y \
    libopenblas0 \
    && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --shell /bin/bash ensemble
USER ensemble
WORKDIR /app

# Copiar desde builder
COPY --from=builder --chown=ensemble:ensemble /usr/local/lib/
   python3.11/site-packages/ /usr/local/lib/python3.11/site-
   packages/
COPY --from=builder --chown=ensemble:ensemble /usr/local/bin/ /
   usr/local/bin/
```

```
COPY --from=builder --chown=ensemble:ensemble /app/ /app/

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --
   retries=3 \
    CMD python -c "import␣src.ensemble;␣print('OK')" || exit 1

# Comando por defecto
CMD ["python", "-m", "src.api.main"]
```

12.2. API REST Completa

```
from fastapi import FastAPI, HTTPException, BackgroundTasks,
   Depends
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import HTTPBearer,
   HTTPAuthorizationCredentials
from pydantic import BaseModel, Field, validator
from typing import List, Dict, Optional, Any
import numpy as np
import pandas as pd
import joblib
import logging
from datetime import datetime, timedelta

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = FastAPI(
    title="Spatial-Temporal␣Ensemble␣Forecaster",
    description="Advanced␣ensemble␣forecasting␣with␣spatial-
       temporal␣dependencies",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc"
)

# Middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

security = HTTPBearer()

# Modelos Pydantic
```

```
36 class SpatialLocation(BaseModel):
37     region_id: int = Field(..., description="Unique region
          identifier")
38     latitude: float = Field(..., ge=-90, le=90, description="
          Latitude coordinate")
39     longitude: float = Field(..., ge=-180, le=180, description="
          Longitude coordinate")
40 
41 class ForecastRequest(BaseModel):
42     regions: List[SpatialLocation] = Field(..., description="
          Spatial locations")
43     features: Dict[str, List[float]] = Field(..., description="
          Input features by variable name")
44     horizon: int = Field(..., gt=0, le=24, description="Forecast
          horizon")
45     confidence_level: float = Field(0.95, ge=0.5, le=0.99,
          description="Confidence level for intervals")
46     include_explanations: bool = Field(False, description="
          Include model explanations")
47 
48     @validator('features')
49     def validate_features(cls, v, values):
50         if 'regions' in values:
51             n_regions = len(values['regions'])
52             for feature_name, feature_values in v.items():
53                 if len(feature_values) != n_regions:
54                     raise ValueError(f"Feature '{feature_name}'
                        must have {n_regions} values")
55         return v
56 
57 class PredictionResult(BaseModel):
58     region_id: int
59     predictions: List[float] = Field(..., description="Forecast
          values")
60     confidence_intervals: List[Dict[str, float]] = Field(...,
          description="Confidence intervals")
61     spatial_effects: Optional[Dict[str, float]] = None
62 
63 class ForecastResponse(BaseModel):
64     request_id: str = Field(..., description="Unique request
          identifier")
65     timestamp: datetime = Field(..., description="Processing
          timestamp")
66     results: List[PredictionResult] = Field(..., description="
          Forecast results by region")
67     model_metadata: Dict[str, Any] = Field(..., description="
          Model information")
68     performance_metrics: Optional[Dict[str, float]] = None
69     explanations: Optional[Dict[str, Any]] = None
70 
71 class ModelStatus(BaseModel):
```

```
    model_version: str
    last_training: datetime
    training_data_size: int
    performance_metrics: Dict[str, float]
    drift_status: Dict[str, Any]
    health_check: Dict[str, bool]

# Dependencias
def get_model():
    """Dependency para cargar el modelo."""
    try:
        model = joblib.load('models/ensemble_model.pkl')
        return model
    except Exception as e:
        logger.error(f"Error loading model: {e}")
        raise HTTPException(status_code=500, detail="Model
            loading failed")

def authenticate_token(credentials: HTTPAuthorizationCredentials
    = Depends(security)):
    """Autenticaci n simple por token."""
    # En producci n, usar JWT y validaci n real
    if credentials.credentials != "your-secret-token":
        raise HTTPException(status_code=401, detail="Invalid
            authentication token")
    return credentials.credentials

# Variables globales para caching
model_cache = None
last_model_load = None
MODEL_CACHE_TTL = timedelta(hours=1)

async def get_cached_model():
    """Modelo con caching para mejor performance."""
    global model_cache, last_model_load

    current_time = datetime.now()
    if (model_cache is None or
        last_model_load is None or
        current_time - last_model_load > MODEL_CACHE_TTL):

        try:
            model_cache = joblib.load('models/ensemble_model.pkl'
                )
            last_model_load = current_time
            logger.info("Model loaded/reloaded from disk")
        except Exception as e:
            logger.error(f"Error loading model: {e}")
            raise HTTPException(status_code=500, detail="Model
                loading failed")

```

```
    return model_cache

# Endpoints principales
@app.get("/health")
async def health_check():
    """Health check endpoint."""
    try:
        model = await get_cached_model()
        return {
            "status": "healthy",
            "timestamp": datetime.now(),
            "model_loaded": model is not None,
            "version": "1.0.0"
        }
    except Exception as e:
        return {
            "status": "unhealthy",
            "error": str(e),
            "timestamp": datetime.now()
        }

@app.post("/forecast", response_model=ForecastResponse)
async def generate_forecast(
    request: ForecastRequest,
    background_tasks: BackgroundTasks,
    token: str = Depends(authenticate_token)
):
    """Endpoint principal para generar forecasts."""

    request_id = f"req_{datetime.now().strftime('%Y%m%d_%H%M%S')}
       _{np.random.randint(1000, 9999)}"
    logger.info(f"Processing forecast request {request_id}")

    try:
        # Cargar modelo
        model = await get_cached_model()

        # Preparar datos de entrada
        n_regions = len(request.regions)
        feature_matrix = np.zeros((n_regions, len(request.
           features)))

        for i, (feature_name, feature_values) in enumerate(
           request.features.items()):
            feature_matrix[:, i] = feature_values

        # Construir matriz espacial si es necesario
        if hasattr(model, 'spatial_weights') and model.
           spatial_weights is not None:
            coordinates = np.array([[loc.latitude, loc.longitude]
                for loc in request.regions])
```

```
164             # Usar coordenadas para efectos espaciales
165
166         # Generar predicciones
167         start_time = datetime.now()
168         predictions = model.predict(feature_matrix)
169         processing_time = (datetime.now() - start_time).
                total_seconds()
170
171         # Calcular intervalos de confianza si el modelo los
                soporta
172         confidence_intervals = []
173         if hasattr(model, 'predict_with_uncertainty'):
174             pred_mean, pred_lower, pred_upper = model.
                    predict_with_uncertainty(
175                 feature_matrix, confidence_level=request.
                        confidence_level)
176
177             for i in range(len(predictions)):
178                 confidence_intervals.append({
179                     'lower': float(pred_lower[i]),
180                     'upper': float(pred_upper[i])
181                 })
182         else:
183             # Intervalos simplificados basados en error
                    hist rico
184             pred_std = getattr(model, 'prediction_std_', np.std(
                    predictions) * 0.1)
185             z_score = 1.96 if request.confidence_level == 0.95
                    else 2.58
186
187             for pred in predictions:
188                 margin = z_score * pred_std
189                 confidence_intervals.append({
190                     'lower': float(pred - margin),
191                     'upper': float(pred + margin)
192                 })
193
194         # Construir resultados
195         results = []
196         for i, region in enumerate(request.regions):
197             # Predicciones multi-step si horizon > 1
198             if request.horizon == 1:
199                 region_predictions = [float(predictions[i])]
200                 region_intervals = [confidence_intervals[i]]
201             else:
202                 # Implementar predicci n multi-step
203                 region_predictions = []
204                 region_intervals = []
205
206                 current_features = feature_matrix[i:i+1]  #
                        Single region
```

```
207
208                 for h in range(request.horizon):
209                     step_pred = model.predict(current_features)
                            [0]
210                     region_predictions.append(float(step_pred))
211
212                     # Intervalo para este step
213                     margin = z_score * pred_std * np.sqrt(h + 1)
                            # Increasing uncertainty
214                     region_intervals.append({
215                         'lower': float(step_pred - margin),
216                         'upper': float(step_pred + margin)
217                     })
218
219                     # Actualizar features para siguiente step (
                            auto-regressive)
220                     if hasattr(model, '
                            update_features_for_next_step'):
221                         current_features = model.
                                update_features_for_next_step(
222                             current_features, step_pred)
223
224             results.append(PredictionResult(
225                 region_id=region.region_id,
226                 predictions=region_predictions,
227                 confidence_intervals=region_intervals
228             ))
229
230         # Metadata del modelo
231         model_metadata = {
232             'model_type': model.__class__.__name__,
233             'n_base_models': getattr(model, 'n_models_', 1),
234             'training_features': getattr(model, 'feature_names_',
                    list(request.features.keys())),
235             'processing_time_seconds': processing_time
236         }
237
238         # Agregar explicaciones si se solicitan
239         explanations = None
240         if request.include_explanations and hasattr(model, '
                get_model_importances'):
241             explanations = {
242                 'model_importances': model.get_model_importances
                        (),
243                 'feature_importances': getattr(model, '
                        feature_importances_', {})
244             }
245
246         # Log de la transacci n en background
247         background_tasks.add_task(
248             log_forecast_transaction,
```

```
            request_id, n_regions, request.horizon,
                processing_time
        )

        response = ForecastResponse(
            request_id=request_id,
            timestamp=datetime.now(),
            results=results,
            model_metadata=model_metadata,
            explanations=explanations
        )

        logger.info(f"Request {request_id} completed successfully")
        return response

    except Exception as e:
        logger.error(f"Error processing request {request_id}: {e}")
        raise HTTPException(
            status_code=500,
            detail=f"Forecast generation failed: {str(e)}"
        )

@app.get("/model/status", response_model=ModelStatus)
async def get_model_status(token: str = Depends(
    authenticate_token)):
    """Obtener estado del modelo."""

    try:
        model = await get_cached_model()

        # Informaci n del modelo
        model_info = {
            'model_version': getattr(model, 'version_', '1.0.0'),
            'last_training': getattr(model, 'training_timestamp_'
                , datetime.now()),
            'training_data_size': getattr(model, 'training_size_'
                , 0),
            'performance_metrics': getattr(model, '
                performance_metrics_', {}),
            'drift_status': getattr(model, 'drift_status_', {'
                status': 'unknown'}),
            'health_check': {
                'model_loaded': True,
                'can_predict': hasattr(model, 'predict'),
                'has_spatial_weights': hasattr(model, '
                    spatial_weights')
            }
        }

```

```
291         return ModelStatus(**model_info)
292
293     except Exception as e:
294         logger.error(f"Error getting model status: {e}")
295         raise HTTPException(status_code=500, detail=str(e))
296
297 @app.post("/model/retrain")
298 async def trigger_retraining(
299     background_tasks: BackgroundTasks,
300     token: str = Depends(authenticate_token)
301 ):
302     """Trigger manual retraining."""
303
304     retrain_id = f"retrain_{datetime.now().strftime('%Y%m%d_%H%M%
        S')}"
305
306     # Ejecutar reentrenamiento en background
307     background_tasks.add_task(perform_model_retraining,
        retrain_id)
308
309     return {
310         "message": "Retraining initiated",
311         "retrain_id": retrain_id,
312         "estimated_completion": datetime.now() + timedelta(hours
            =2)
313     }
314
315 # Funciones de background
316 async def log_forecast_transaction(request_id: str, n_regions:
    int,
317                                    horizon: int, processing_time:
                                       float):
318     """Log transaction para monitoreo."""
319     log_entry = {
320         'request_id': request_id,
321         'timestamp': datetime.now(),
322         'n_regions': n_regions,
323         'horizon': horizon,
324         'processing_time': processing_time
325     }
326
327     # En producci n: enviar a sistema de logging/monitoring
328     logger.info(f"Transaction logged: {log_entry}")
329
330 async def perform_model_retraining(retrain_id: str):
331     """Reentrenar modelo en background."""
332     try:
333         logger.info(f"Starting retraining {retrain_id}")
334
335         # Cargar nuevos datos
336         # new_data = load_new_training_data()
```

```
337
338         # Reentrenar modelo
339         # retrained_model = retrain_ensemble_model(new_data)
340
341         # Validar modelo
342         # validation_results = validate_retrained_model(
                retrained_model)
343
344         # Guardar si es mejor
345         # if validation_results['improved']:
346         #     save_model(retrained_model, f'models/
                ensemble_model_{retrain_id}.pkl')
347         #
348         #     # Atomic swap del modelo activo
349         #     os.rename(f'models/ensemble_model_{retrain_id}.pkl
                ',
350         #               'models/ensemble_model.pkl')
351
352         logger.info(f"Retraining␣{retrain_id}␣completed␣
                successfully")
353
354     except Exception as e:
355         logger.error(f"Retraining␣{retrain_id}␣failed:␣{e}")
356
357 if __name__ == "__main__":
358     import uvicorn
359     uvicorn.run(app, host="0.0.0.0", port=8000)
```

13. Bibliografia

Referencias

[1] Anselin, L. (1988). *Spatial Econometrics: Methods and Models*. Kluwer Academic Publishers, Dordrecht.

[2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[3] Brockwell, P. J., & Davis, R. A. (2016). *Introduction to Time Series and Forecasting*. 3rd edition, Springer, New York.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[5] Cliff, A. D., & Ord, J. K. (1981). *Spatial Processes: Models and Applications*. Pion, London.

[6] Elhorst, J. P. (2003). Specification and estimation of spatial panel data models. *International Regional Science Review*, 26(3), 244–268.

[7] Elhorst, J. P. (2014). *Spatial Econometrics: From Cross-Sectional Data to Spatial Panels*. Springer, Berlin.

[8] Grimmett, G., & Stirzaker, D. (2020). *Probability and Random Processes*. 4th edition, Oxford University Press, Oxford.

[9] Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press, Princeton.

[10] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[11] Hospedales, T., Antoniou, A., Micaelli, P., & Storkey, A. (2021). Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9), 5149–5169.

[12] Johansen, S. (1995). *Likelihood-Based Inference in Cointegrated Vector Autoregressive Models*. Oxford University Press, Oxford.

[13] LeSage, J., & Pace, R. K. (2009). *Introduction to Spatial Econometrics*. CRC Press, Boca Raton.

[14] Lütkepohl, H. (2005). *New Introduction to Multiple Time Series Analysis*. Springer, Berlin.

[15] Oreshkin, B. N., Carpov, D., Chapados, N., & Bengio, Y. (2019). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *arXiv preprint arXiv:1905.10437*.

[16] Ross, S. M. (2014). *Introduction to Probability Models*. 11th edition, Academic Press, Boston.

[17] Strang, G. (2016). *Introduction to Linear Algebra*. 5th edition, Wellesley-Cambridge Press, Wellesley.

[18] Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM, Philadelphia.

[19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[20] Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259.

[21] Yang, Y. (2004). Combining forecasting procedures: some theoretical results. *Econometric Theory*, 20(1), 176–222.

[22] Yu, J., De Jong, R., & Lee, L. F. (2008). Quasi-maximum likelihood estimators for spatial dynamic panel data with fixed effects when both n and T are large. *Journal of Econometrics*, 146(1), 118–134.

14. Anexo A: Checklist de Implementación

14.1. Checklist Técnico Obligatorio

Infraestructura Base:

☐ Repositorio con estructura definida implementada

☐ CI/CD pipeline configurado (GitHub Actions/GitLab CI)

☐ Pre-commit hooks funcionando (black, isort, flake8, mypy)

☐ Docker containers para dev/prod funcionales

☐ Documentación automática generándose (Sphinx)

☐ Tests unitarios con coverage ¿90 %

☐ Tests de integración implementados

☐ Logging estructurado configurado

Modelos Base:

☐ ARIMA con estimación ML completa

☐ VAR con tests de cointegración (Johansen)

☐ VECM si cointegración detectada

☐ SAR con estimación ML estable

☐ SEM con manejo de singularidades

☐ Panel espacial dinámico funcional

☐ Random Forest con features espaciales

☐ XGBoost con regularización temporal

☐ LSTM espacio-temporal

☐ Transformer con atención espacial

Ensemble Sistema:

☐ Ensemble estático con optimización convexa

☐ Ensemble dinámico con online learning

☐ Stacking con meta-learner robusto

☐ Validación cruzada temporal sin data leakage

☐ Optimización bayesiana de hiperparámetros

☐ Regularización espacial implementada

☐ Uncertainty quantification funcional

☐ Feature importance y explicabilidad

Validación y Testing:

☐ PurgedTimeSeriesSplit implementado correctamente

☐ SpatialBlockCV para validación espacial

☐ Tests de Diebold-Mariano funcionando

☐ Model Confidence Set implementado

☐ Reality Check test para comparación múltiple

☐ Tests de drift multivariado

☐ Sistema de alertas y monitoring

☐ Acceptance tests automatizados

Production Ready:

☐ API REST completa con documentación OpenAPI

☐ Autenticación y autorización implementadas

☐ Rate limiting configurado

☐ Health checks funcionando

☐ Logging estructurado para producción

☐ Monitoring y métricas (Prometheus/Grafana)

☐ Backup y recovery procedures

☐ Load testing completado

14.2. Criterios de Calidad No Negociables

Código:

$$\text{Type coverage} = 100\,\% \text{ en API pública} \tag{37}$$

$$\text{Test coverage} \geq 95\,\% \text{ líneas de código} \tag{38}$$

$$\text{Cyclomatic complexity} < 10 \text{ por función} \tag{39}$$

$$\text{Duplicación de código} < 3\,\% \tag{40}$$

$$\text{Tiempo build} < 5 \text{ minutos} \tag{41}$$

Performance:

$$\text{Latencia API} < 100\text{ms (p95)} \tag{42}$$
$$\text{Throughput API} > 1000\text{ req/min} \tag{43}$$
$$\text{Memory footprint} < 4\text{GB (baseline)} \tag{44}$$
$$\text{Training time} < 2\text{h (1M observations)} \tag{45}$$
$$\text{Model loading time} < 10\text{s} \tag{46}$$

Accuracy:

$$\text{RMSE improvement} > 15\,\% \text{ vs naive} \tag{47}$$
$$\text{RMSE improvement} > 8\,\% \text{ vs best individual} \tag{48}$$
$$\text{Directional accuracy} > 60\,\% \tag{49}$$
$$\text{Spatial coherence} > 0{,}75 \tag{50}$$
$$\text{Temporal consistency} < 0{,}1 \text{ correlation en residuos} \tag{51}$$

15. Anexo B: Cronograma Detallado

15.1. Timeline Realista (20 semanas)

Fase 0: Preparación (Semana 0)

- Setup de entorno de desarrollo completo
- Instalación y configuración de herramientas
- Creación de repositorio con estructura inicial
- Setup de CI/CD básico

Fase I: Fundamentos (Semanas 1-4)

- **Semana 1:** Álgebra lineal + implementaciones básicas
- **Semana 2:** Procesos estocásticos + tests estadísticos
- **Semana 3:** ARIMA completo + diagnósticos residuales
- **Semana 4:** VAR/VECM + tests de cointegración

Fase II: Modelos Espaciales (Semanas 5-7)

- **Semana 5:** Matrices espaciales + tests de dependencia
- **Semana 6:** SAR/SEM con estimación ML robusta
- **Semana 7:** Panel espacial dinámico + validación

Fase III: Machine Learning (Semanas 8-10)

- **Semana 8:** Random Forest + XGBoost espacial
- **Semana 9:** LSTM espacio-temporal + Transformer

- **Semana 10:** N-BEATS modificado + validación

Fase IV: Ensemble Avanzado (Semanas 11-13)

- **Semana 11:** Stacking + meta-learner espacial
- **Semana 12:** Online learning + ensemble dinámico
- **Semana 13:** Optimización bayesiana + regularización

Fase V: Validación Completa (Semanas 14-16)

- **Semana 14:** Validación cruzada temporal + espacial
- **Semana 15:** Tests estadísticos + benchmarking
- **Semana 16:** Drift detection + monitoring system

Fase VI: Productización (Semanas 17-20)

- **Semana 17:** API REST + documentación completa
- **Semana 18:** Containerización + deployment pipeline
- **Semana 19:** Load testing + optimization
- **Semana 20:** Acceptance testing + documentation final

15.2. Hitos Críticos y Gates de Calidad

Gate 1 (Final Semana 4): Fundamentos matemáticos

- ARIMA funcional con tests residuales pasando
- VAR con matriz de cointegración correcta
- Tests unitarios ¿85 % coverage
- **Criterio de paso:** Reproducir resultados de papers conocidos

Gate 2 (Final Semana 7): Modelos espaciales operativos

- SAR convergiendo en datasets sintéticos
- Estimador ML numéricamente estable
- Tests de autocorrelación espacial funcionando
- **Criterio de paso:** Replicar resultados Anselin (1988)

Gate 3 (Final Semana 10): ML models integrados

- Todos los modelos ML entrenando correctamente
- Features espaciales mejorando performance
- Pipeline de entrenamiento automatizado

- **Criterio de paso:** Beating baselines en 3+ datasets

Gate 4 (Final Semana 13): Ensemble completo

- Meta-learner optimizando pesos correctamente
- Ensemble superando modelos individuales consistentemente
- Uncertainty quantification calibrada
- **Criterio de paso:** ¿10 % improvement vs best individual

Gate 5 (Final Semana 16): Validación científica

- Tests estadísticos confirmando significancia
- Validación temporal sin data leakage
- Drift detection funcional
- **Criterio de paso:** Acceptance tests ¿90 % pass rate

Gate Final (Final Semana 20): Production ready

- API cumpliendo SLA de latencia
- Load testing pasando
- Monitoring y alertas operativas
- **Criterio de paso:** Deployment exitoso en staging

16. Anexo C: Resources y Budget

16.1. Presupuesto Detallado

Hardware (One-time):

- Workstation desarrollo: $5,000 USD
- GPU NVIDIA RTX 4090: $1,600 USD
- Storage adicional (4TB NVMe): $800 USD
- **Total Hardware: $7,400 USD**

Software y Servicios (Anual):

- Cloud compute credits (AWS/GCP): $6,000 USD
- Software licenses (IDEs, etc.): $1,200 USD
- CI/CD platform (GitHub Actions): $500 USD
- Monitoring tools (DataDog/NewRelic): $2,400 USD
- **Total Software: $10,100 USD/año**

Formación y Literatura:

- Libros técnicos y papers: $800 USD
- Online courses (Coursera, edX): $500 USD
- Conference attendance (opcional): $3,000 USD
- **Total Formación: $4,300 USD**

Total Proyecto: $21,800 USD + tiempo/salario

16.2. ROI Estimado

Valor generado vs alternativas:

- Software comercial equivalente: $50,000-200,000 USD/año
- Consultoría externa: $100,000-300,000 USD
- Contratación de equipo especializado: $400,000+ USD/año

Payback period estimado: 2-6 meses

17. Consideraciones Finales

17.1. Advertencias Críticas

No negociables técnicos:

1. La validación temporal con purging es absolutamente crítica
2. Omitir regularización espacial invalidará resultados
3. Tests de autocorrelación deben pasar en todos los residuos
4. Type hints al 100 % no son opcionales

Riesgos principales:

1. **Data leakage temporal:** Destroza completamente la validez
2. **Overfitting ensemble:** Meta-learner puede sobreajustar
3. **Inestabilidad numérica:** Matrices espaciales mal condicionadas
4. **Drift no detectado:** Performance degradation silenciosa

17.2. Éxito Garantizado

Este proyecto tiene éxito garantizado si y solo si:

$$\text{Tiempo invertido} \geq 800 \text{ horas especializadas} \tag{52}$$

$$\text{Rigor metodológico} = 100\,\% \text{ (no negociable)} \tag{53}$$

$$\text{Validación científica} = \text{Completa y rigurosa} \tag{54}$$

$$\text{Code quality} = \text{Production grade} \tag{55}$$

Cualquier compromiso en estos criterios resultará en un sistema no funcional o científicamente inválido.

El tiempo total estimado de 920 horas es el mínimo absoluto para un resultado profesional. Acelerar este proceso comprometerá irremediablemente la calidad técnica.